DUC meetup/4

Про Kubernetes-ориентированные операционки

и то, к чему нас привели
их ограничения

DUC meetup/4

Денис Романенко

Технический
директор продукта
Cluster and Infrastructure

О чём мы будем сегодня говорить?

DUC meetup/4

Сегодня мы обсудим:

01 Откуда возникла идея специализированных дистрибутивов для Kubernetes.

02 По какому пути шло развитие этих дистрибутивов.

03 Какие общие концепции им присущи.

04 Какие есть достоинства и недостатки.

05 Как красивые идеи не всегда работают в реальной жизни.

06 Какие пути решения проблем в реальной эксплуатации мы применяем.

Эволюция операционных систем с точки зрения Kubernetes

DUC meetup/4

Эволюция ОС с точки зрения Kubernetes

Первоначально кластеры собирали на базе дистрибутивов общего назначения: Centos, Ubuntu и т. д., используя их серверные варианты и устанавливая компоненты Kubernetes дополнительно.

Эволюция ОС с точки зрения Kubernetes

Такой подход имеет свои известные минусы:

Эволюция ОС с точки зрения Kubernetes

Такой подход имеет свои известные минусы:

- Нужно ставить компоненты Kubernetes.

Эволюция ОС с точки зрения Kubernetes

Такой подход имеет свои известные минусы:

- Нужно ставить компоненты Kubernetes.
- Нужно следить за обновлениями.

Эволюция ОС с точки зрения Kubernetes

Такой подход имеет свои известные минусы:

- Нужно ставить компоненты Kubernetes.
- Нужно следить за обновлениями.
- Нужно настраивать систему для оптимальной работы Kubernetes.

Эволюция ОС с точки зрения Kubernetes

Такой подход имеет свои известные минусы:

- Нужно ставить компоненты Kubernetes.
- Нужно следить за обновлениями.
- Нужно настраивать систему для оптимальной работы Kubernetes.
- Большая поверхность атаки.

Эволюция ОС с точки зрения Kubernetes

Такой подход имеет свои известные минусы:

- Нужно ставить компоненты Kubernetes.
- Нужно следить за обновлениями.
- Нужно настраивать систему для оптимальной работы Kubernetes.
- Большая поверхность атаки.
- Неоптимальное использование ресурсов.

Эволюция ОС с точки зрения Kubernetes

Такой подход имеет свои известные минусы:

- Нужно ставить компоненты Kubernetes.
- Нужно следить за обновлениями.
- Нужно настраивать систему для оптимальной работы Kubernetes.
- Большая поверхность атаки.
- Неоптимальное использование ресурсов.
- Шумные результаты сканеров безопасности.

Эволюция ОС с точки зрения Kubernetes

Ещё одним важным моментом являлось то, что дистрибутивы общего пользования создавались для выполнения многих ролей.

Эволюция ОС с точки зрения Kubernetes

Ещё одним важным моментом являлось то, что дистрибутивы общего пользования создавались для выполнения многих ролей:

- Десктоп

Эволюция ОС с точки зрения Kubernetes

Ещё одним важным моментом являлось то, что дистрибутивы общего пользования создавались для выполнения многих ролей:

- Десктоп
- Сервер

Эволюция ОС с точки зрения Kubernetes

Ещё одним важным моментом являлось то, что дистрибутивы общего пользования создавались для выполнения многих ролей:

- Десктоп
- Сервер
- Тонкий клиент

Эволюция ОС с точки зрения Kubernetes

Но ни одна роль в дистрибутивах общего пользования
не предполагала сценария использования дистрибутива
для развёртывания Kubernetes-кластеров.

Эволюция ОС с точки зрения Kubernetes

Логично, что появились мысли о создании специализированного дистрибутива, который был бы более удобен для использования вместе с Kubernetes.

Отличия Kubernetes-дистрибутивов

DUC meetup/4

Отличия Kubernetes-дистрибутивов

- Ориентация на выполнение одной функции — «контейнеровозка».

Отличия Kubernetes-дистрибутивов

- Ориентация на выполнение одной функции — «контейнеровозка».
- Отказ от управления дистрибутивом стандартными средствами — использование API-oriented-решений.

Отличия Kubernetes-дистрибутивов

- Ориентация на выполнение одной функции — «контейнеровозка».
- Отказ от управления дистрибутивом стандартными средствами — использование API-oriented-решений.
- Отказ от использования пакетных менеджеров в пользу обновления всего дистрибутива единомоментно.

Отличия Kubernetes-дистрибутивов

- Ориентация на выполнение одной функции — «контейнеровозка».
- Отказ от управления дистрибутивом стандартными средствами — использование API-oriented-решений.
- Отказ от использования пакетных менеджеров в пользу обновления всего дистрибутива единомоментно.
- Использование преднастроенных ядер.

Отличия Kubernetes-дистрибутивов

- Ориентация на выполнение одной функции — «контейнеровозка».
- Отказ от управления дистрибутивом стандартными средствами — использование API-oriented-решений.
- Отказ от использования пакетных менеджеров в пользу обновления всего дистрибутива единомоментно.
- Использование преднастроенных ядер.
- Read-only rootFS.

Обзор Kubernetes-дистрибутивов

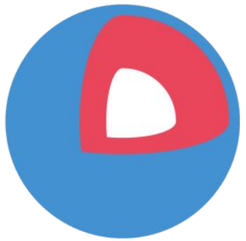
DUC meetup/4

Обзор Kubernetes-дистрибутивов

На текущий момент популярны следующие Kubernetes-native дистрибутивы:

Обзор Kubernetes-дистрибутивов

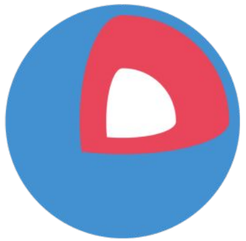
На текущий момент популярны следующие Kubernetes-native дистрибутивы:



Red Hat CoreOS

Обзор Kubernetes-дистрибутивов

На текущий момент популярны следующие Kubernetes-native дистрибутивы:

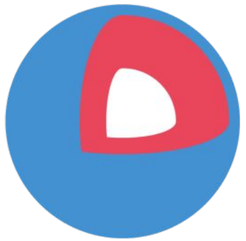


Red Hat CoreOS

Flatcar

Обзор Kubernetes-дистрибутивов

На текущий момент популярны следующие Kubernetes-native дистрибутивы:



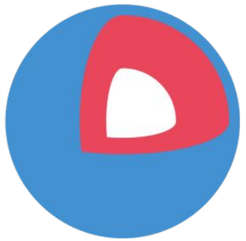
Red Hat CoreOS

Flatcar

K3OS

Обзор Kubernetes-дистрибутивов

На текущий момент популярны следующие Kubernetes-native дистрибутивы:



Red Hat CoreOS

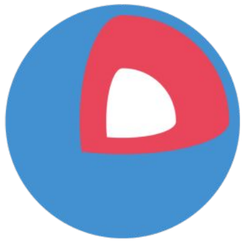
Flatcar

K3OS

Bottlerocket

Обзор Kubernetes-дистрибутивов

На текущий момент популярны следующие Kubernetes-native дистрибутивы:



Red Hat CoreOS

Flatcar

K3OS

Bottlerocket

Talos

Обзор Kubernetes-дистрибутивов

Red Hat CoreOS (Fedora/RHEL):

- В 2018 году компания RedHat купила компанию CoreOS и на базе её наработок и наработок Fedora Atomic Host выпустила Redhat CoreOS. Поставляется как часть OKD. Существует OSS-версия Fedora CoreOS.
- Иммутабельная RootFS на основе RPM-ostree.
- Изоляция скриптов, запускаемых в системе, в контейнеры.
- Ограниченный и контролируемый список установленных пакетов в базовом образе.

Обзор Kubernetes-дистрибутивов

Red Hat CoreOS (Fedora/RHEL):

- Автообновления.
- Акцент на безопасности (SELinux).
- Podman и Docker «из коробки».

Обзор Kubernetes-дистрибутивов

Flatcar:

- Форк CoreOS, разрабатываемый как OSS.
- Иммутабельная RootFS.
- Минимализм: отсутствует пакетный менеджер, нет лишних утилит.
- Конфигурация через ignition: используется специальная утилита ignition для конфигурирования при загрузке, а не cloud-init.

Обзор Kubernetes-дистрибутивов

Flatcar:

- Модульность: поддержка systemd-sysext позволяет расширять функциональность без изменения базовой ОС.
- Бесшовная миграция с CoreOS.
- Акцент на безопасности (SELinux, AppArmor).
- Kubernetes и Docker «из коробки».

Обзор Kubernetes-дистрибутивов

K3OS:

- Иммутабельная RootFS.
- Минимализм: содержит только необходимые компоненты для Kubernetes.
- Kubernetes-native-управление: после старта кластера возможно управление жизненным циклом операционной системы и обновлениями с помощью kubectl.
- Все компоненты control plane запускаются единым бинарным файлом K3s.
- Минимальные требования к ресурсам (1–2 ГБ RAM).
- Kubernetes «из коробки».

Обзор Kubernetes-дистрибутивов

Bottlerocket:

- Иммутабельная RootFS.
- Минимализм: содержит только необходимые компоненты для Kubernetes.
- Ориентирована на AWS.
- SSH-доступ отключён по умолчанию.
- Атомарные обновления.

Обзор Kubernetes-дистрибутивов

Talos — наиболее интересный с точки зрения технологий Kubernetes-native-дистрибутив:

- Иммутабельная RootFS.
- Атомарные обновления с возможностью отката.
- Минимализм: отсутствуют командная оболочка, SSH, пакетный менеджер и лишние стандартные компоненты Linux.

Обзор Kubernetes-дистрибутивов

Talos — наиболее интересный с точки зрения технологий Kubernetes-native-дистрибутив.

- API-центричное управление: вся настройка, обновление и мониторинг осуществляются через API или специализированные CLI-инструменты.
- Полная интеграция с Kubernetes: Talos запускает Kubernetes как встроенную часть ОС.
- Безопасность: реализованы строгие практики безопасности, включая принудительное шифрование, проверку подписей, Mutual TLS, RBAC и рекомендации CIS Benchmark.

Обзор Kubernetes-дистрибутивов

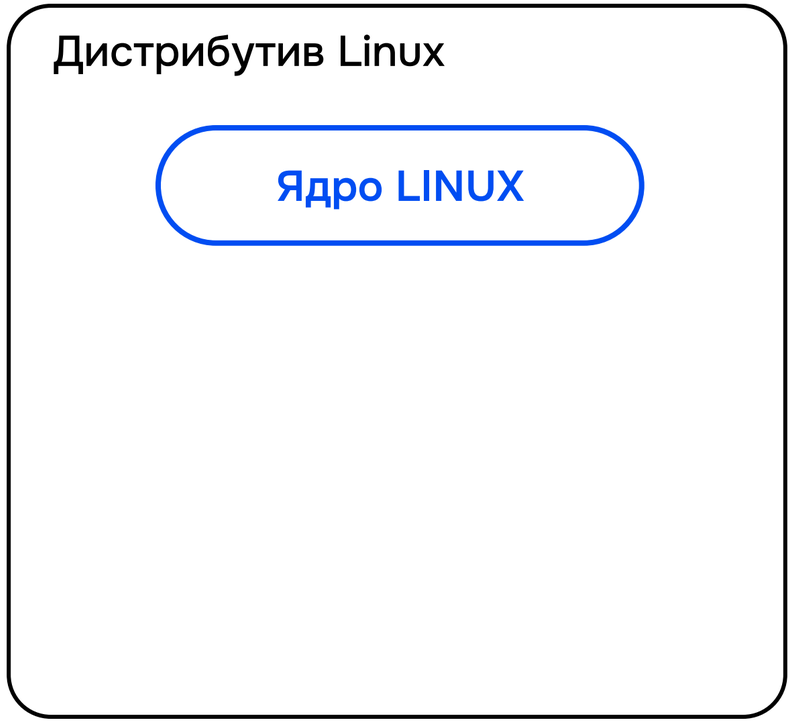
Все дистрибутивы сделаны немного по-разному,
но если посмотреть на базовые принципы, то окажется,
что все дистрибутивы построены примерно по одним принципам:

- Иммутабельность.
- Изоляция ПО.
- Минималистичность.
- Механизмы контейнеризации «из коробки».
- Управление по API.
- Акцент на безопасности.

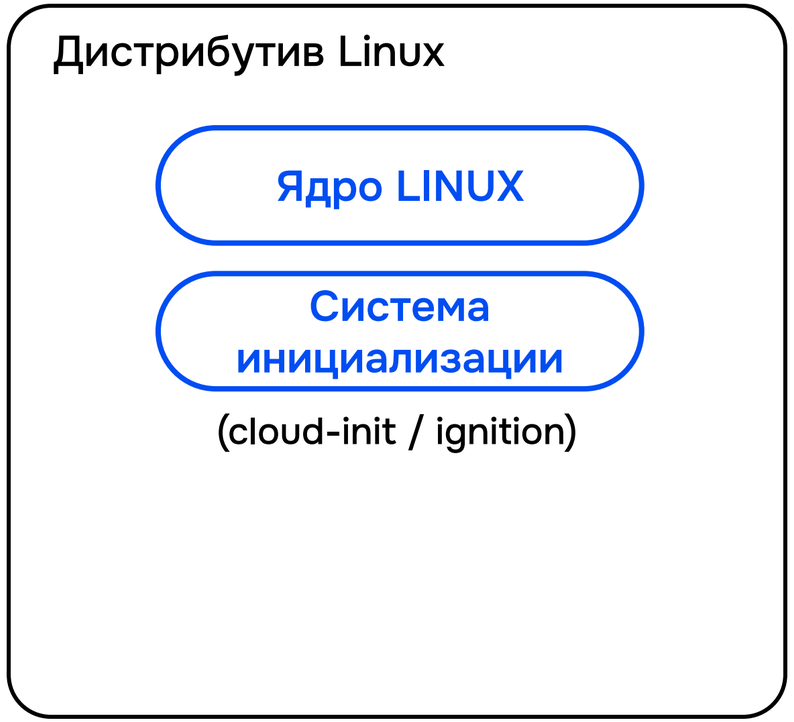
Переход от обычного дистрибутива к Kubernetes-native

DUC meetup/4

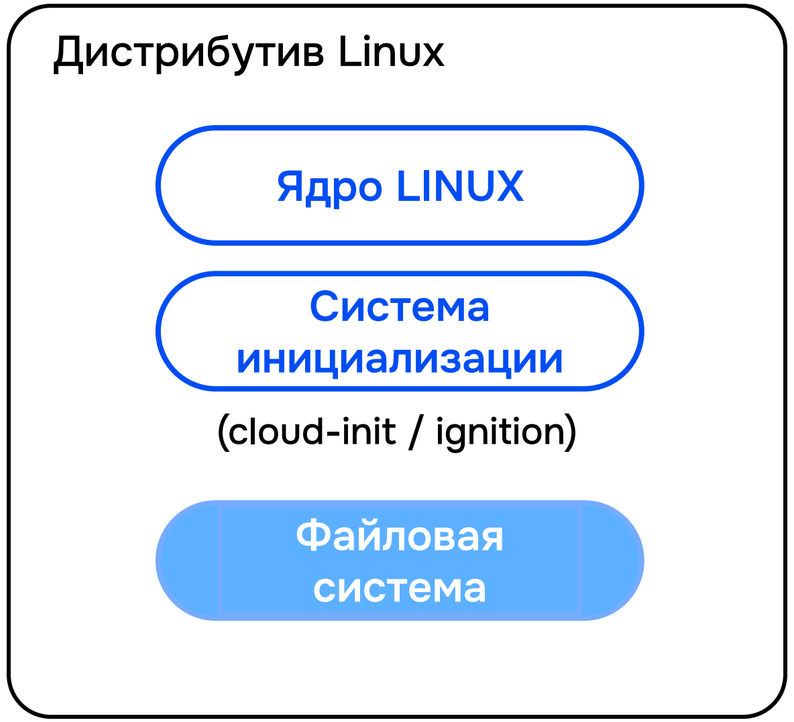
Переход от обычного дистрибутива к Kubernetes-native



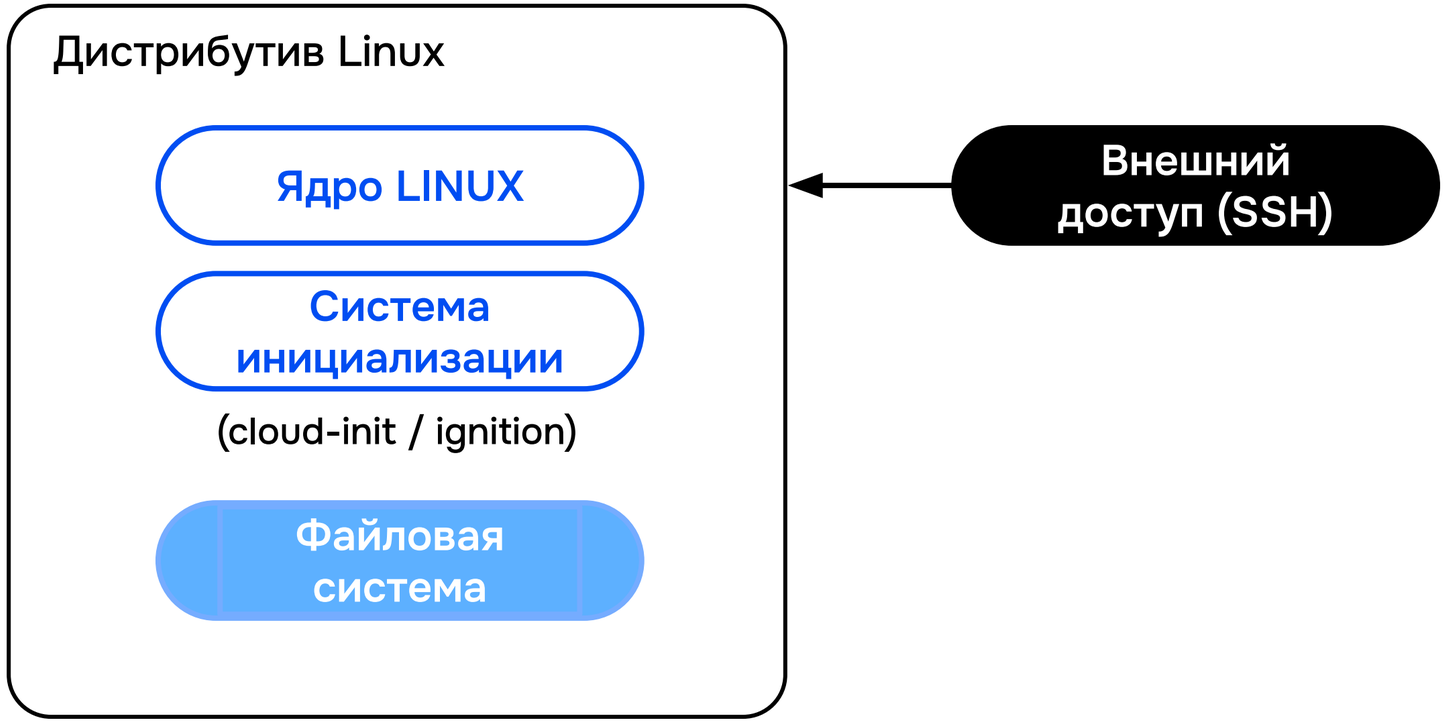
Переход от обычного дистрибутива к Kubernetes-native



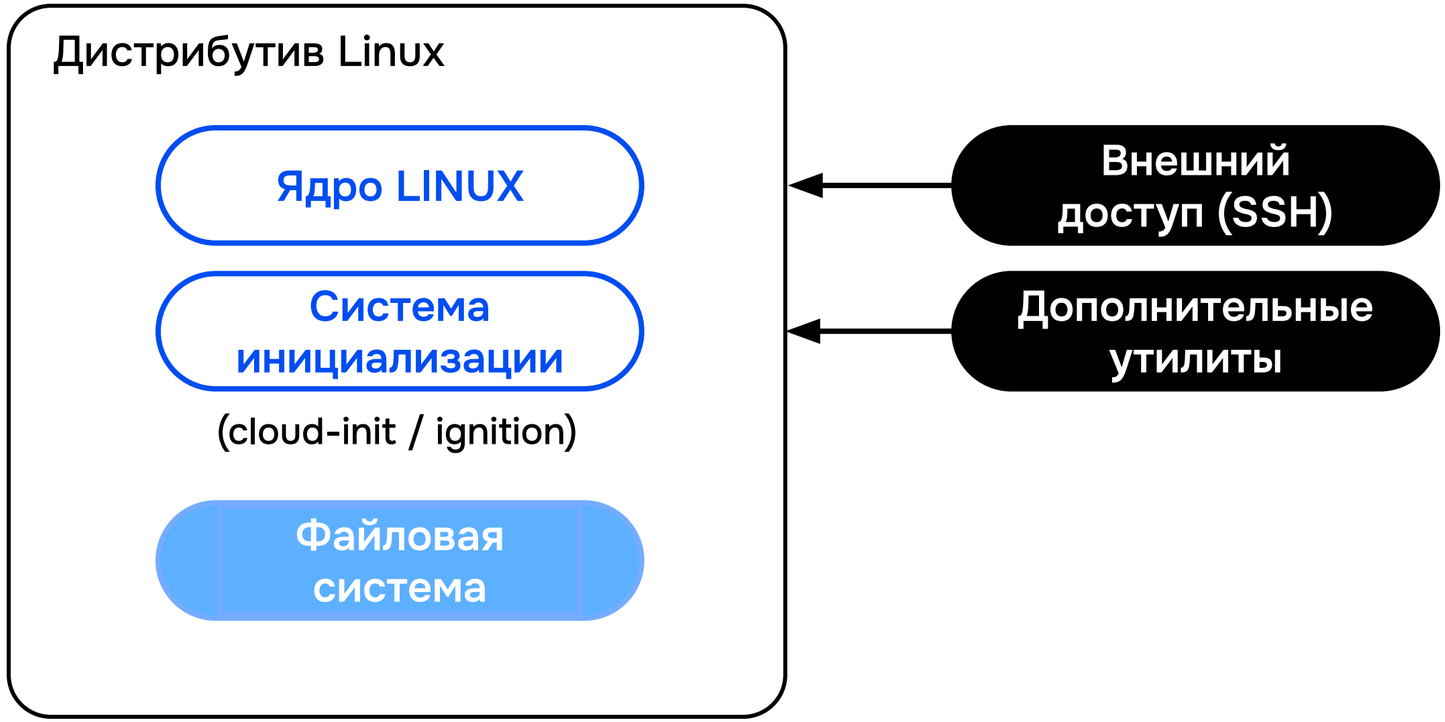
Переход от обычного дистрибутива к Kubernetes-native



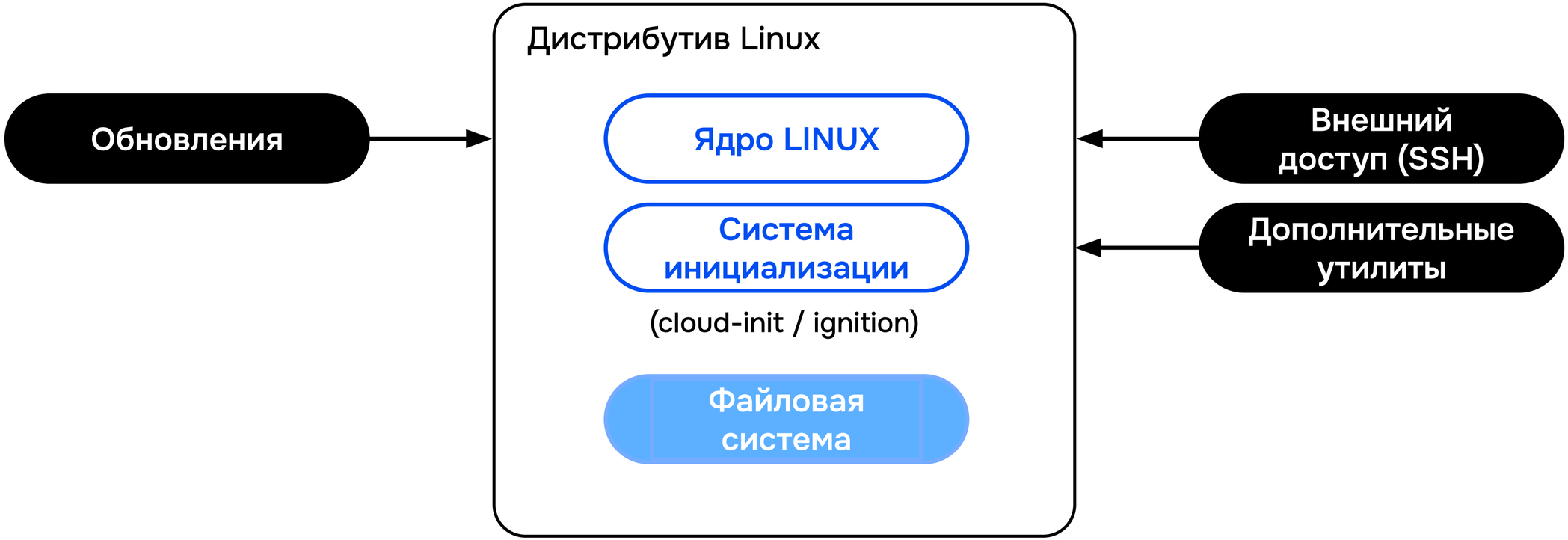
Переход от обычного дистрибутива к Kubernetes-native



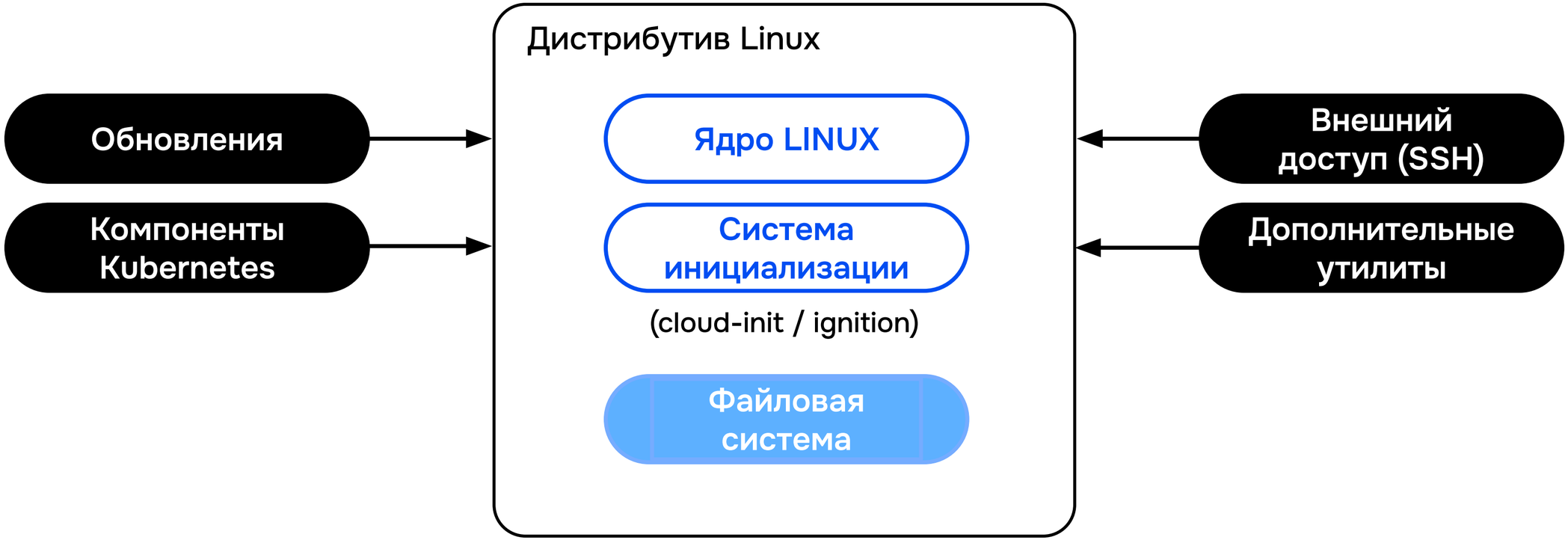
Переход от обычного дистрибутива к Kubernetes-native



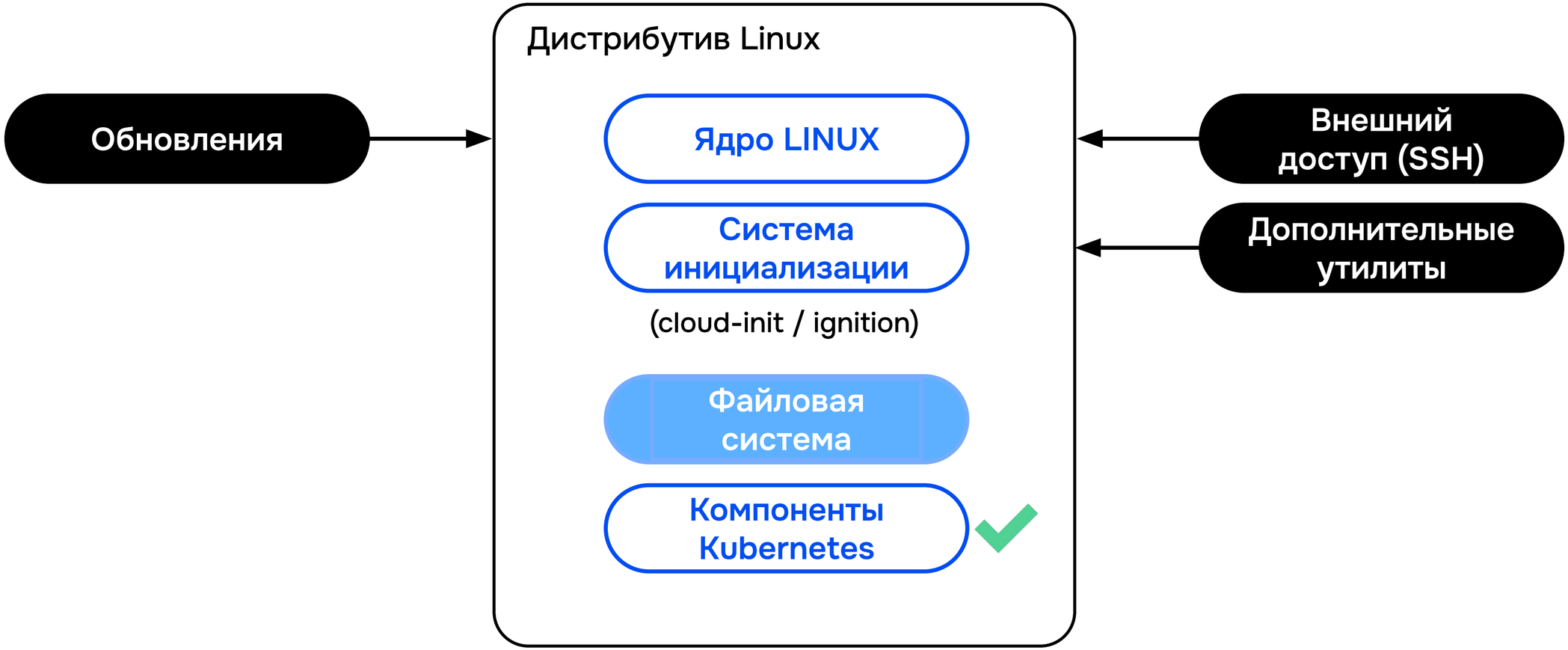
Переход от обычного дистрибутива к Kubernetes-native



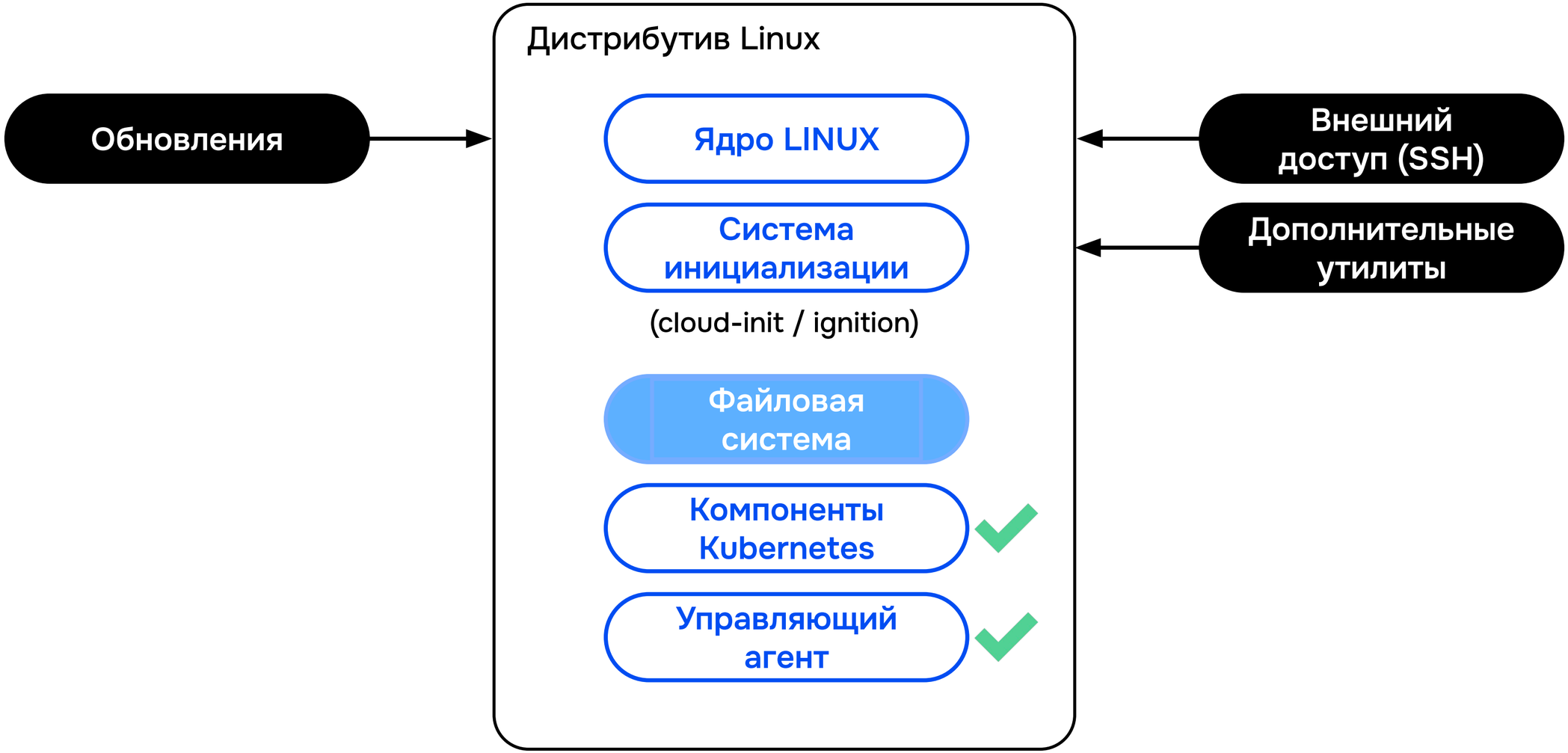
Переход от обычного дистрибутива к Kubernetes-native



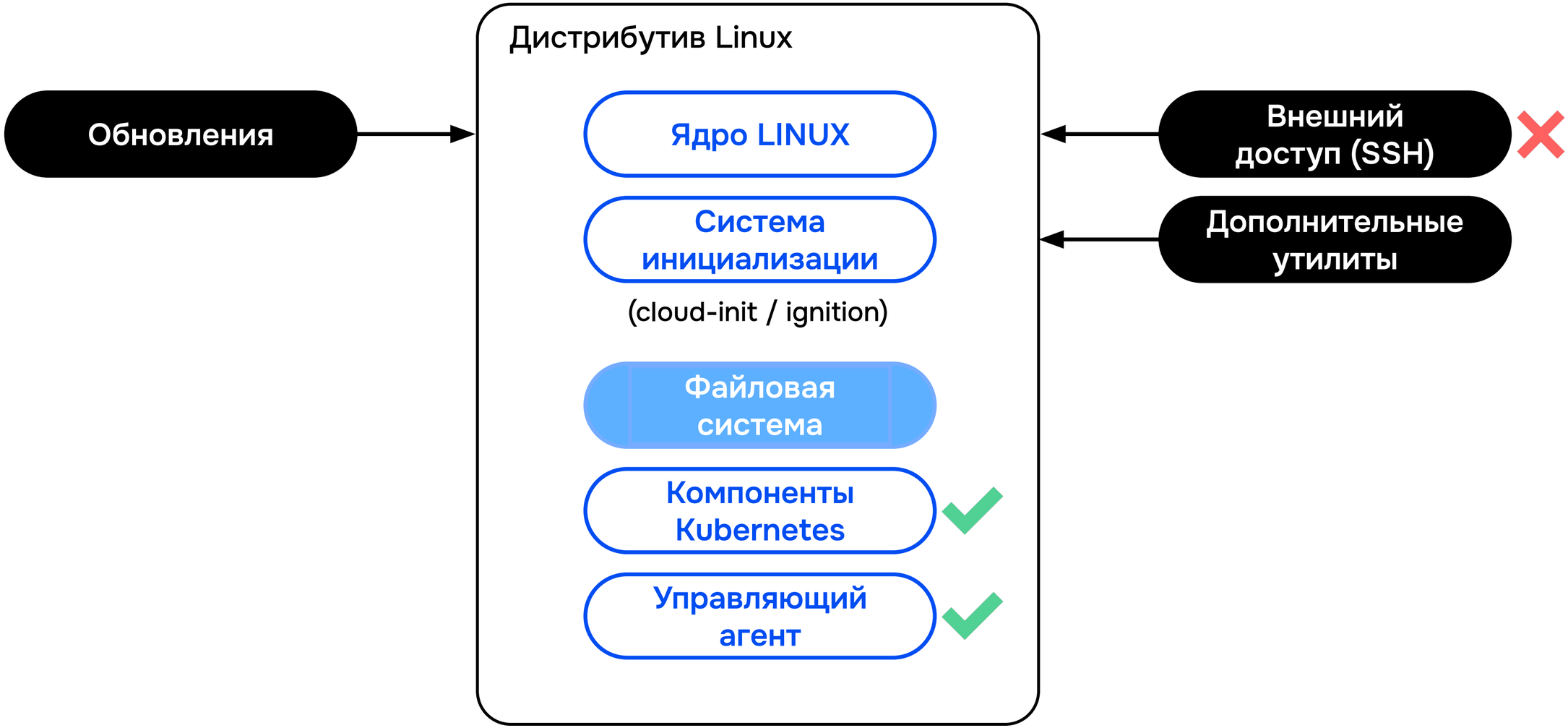
Переход от обычного дистрибутива к Kubernetes-native



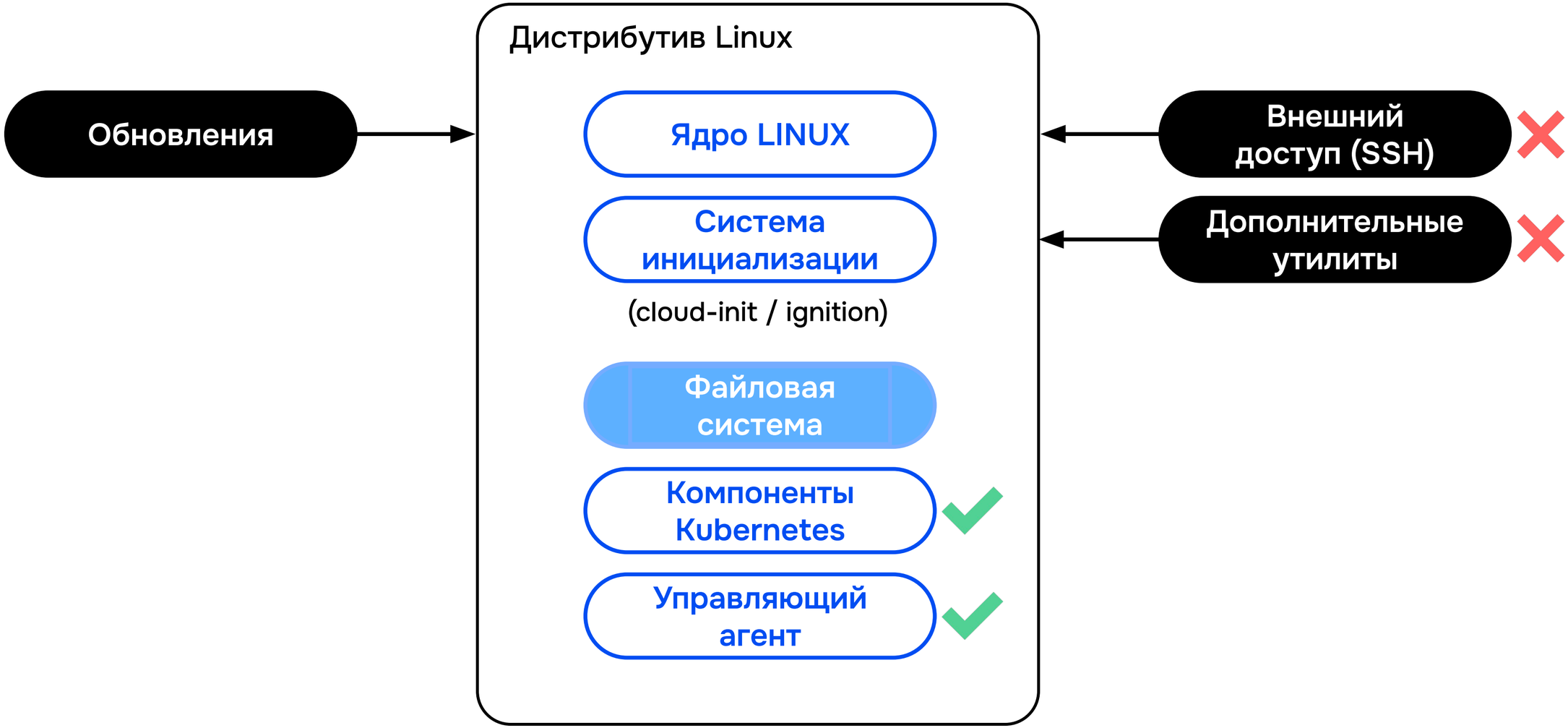
Переход от обычного дистрибутива к Kubernetes-native



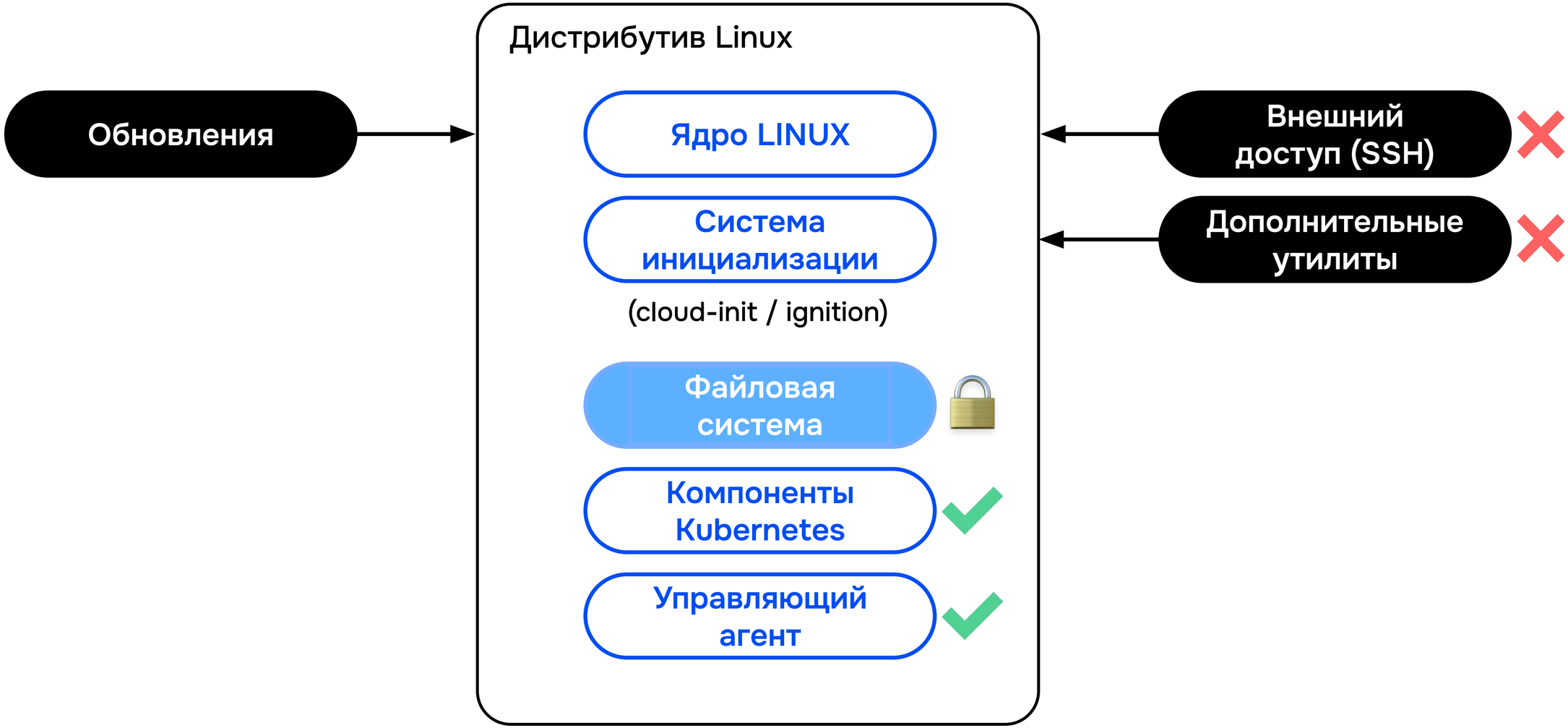
Переход от обычного дистрибутива к Kubernetes-native



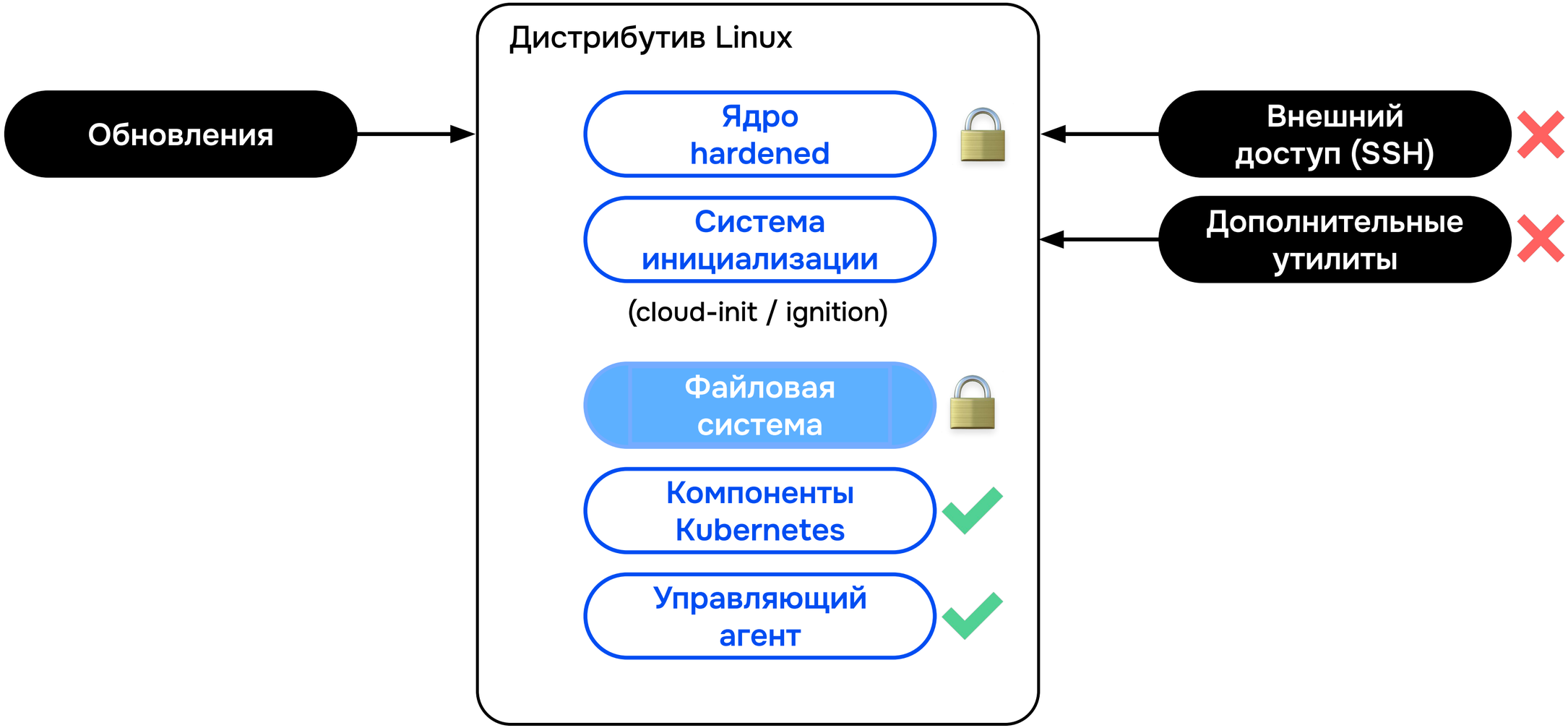
Переход от обычного дистрибутива к Kubernetes-native



Переход от обычного дистрибутива к Kubernetes-native



Переход от обычного дистрибутива к Kubernetes-native



По сути, **Kubernetes-ориентированная ОС** –
это дистрибутив, в котором:

- вычистили всё, не связанное с запуском контейнерных нагрузок а так же оставили минимально необходимый набор для управления;
- предустановили компоненты CRI и Kubernetes;
- сделали ОС иммутабельной.

Плюсы Kubernetes-дистрибутивов

DUC meetup/4

Плюсы Kubernetes-дистрибутивов

Какие же плюсы приносят подобные изменения?

Плюсы Kubernetes-дистрибутивов

Какие же плюсы приносят подобные изменения?

- Мало компонентов = малая поверхность атаки.

Плюсы Kubernetes-дистрибутивов

Какие же плюсы приносят подобные изменения?

- Мало компонентов = малая поверхность атаки.
- Отключена возможность внешнего входа = сложнее проникнуть в систему.

Плюсы Kubernetes-дистрибутивов

Какие же плюсы приносят подобные изменения?

- Мало компонентов = малая поверхность атаки.
- Отключена возможность внешнего входа = сложнее проникнуть в систему.
- Иммутабельная ФС = невозможно закрепиться в системе.

Плюсы Kubernetes-дистрибутивов

Какие же плюсы приносят подобные изменения?

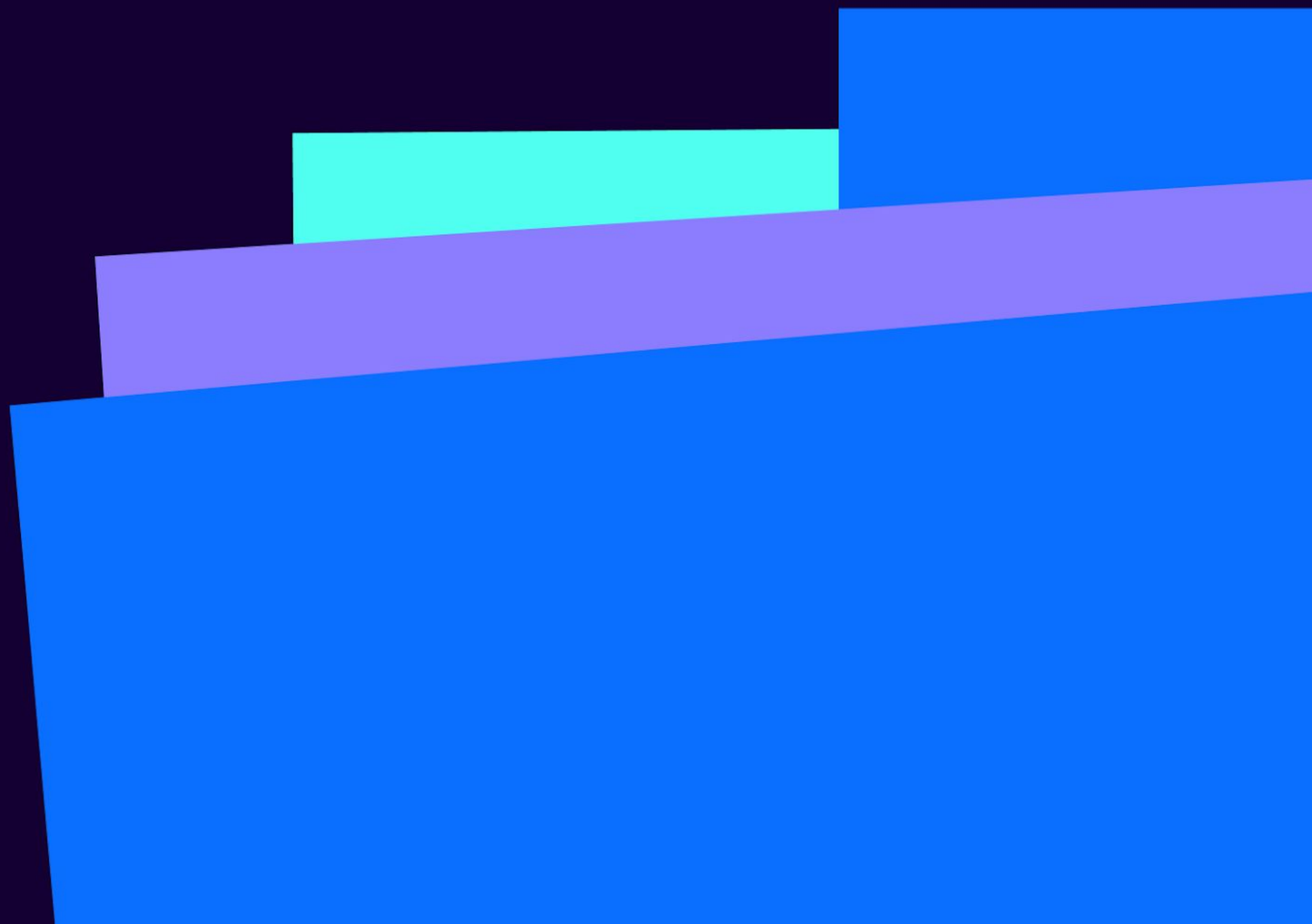
- Мало компонентов = малая поверхность атаки.
- Отключена возможность внешнего входа = сложнее проникнуть в систему.
- Иммутабельная ФС = невозможно закрепиться в системе.
- Предустановлены компоненты K8s = система настроена «из коробки» на максимальную производительность.

Плюсы Kubernetes-дистрибутивов

Какие же плюсы приносят подобные изменения?

- Мало компонентов = малая поверхность атаки.
- Отключена возможность внешнего входа = сложнее проникнуть в систему.
- Иммутабельная ФС = невозможно закрепиться в системе.
- Предустановлены компоненты K8s = система настроена «из коробки» на максимальную производительность.
- Малый размер ОС.

Минусы Kubernetes-дистрибутивов



DUC meetup/4

Минусы Kubernetes-дистрибутивов

Идеала не существует. Любые ограничения, помимо безопасности, приносят также и сложности эксплуатации.

Минусы Kubernetes-дистрибутивов

Идеала не существует. Любые ограничения, помимо безопасности, приносят также и сложности эксплуатации.

- ✕ Мало компонентов = невозможно установить на нестандартное железо.

Минусы Kubernetes-дистрибутивов

Идеала не существует. Любые ограничения, помимо безопасности, приносят также и сложности эксплуатации.

- Мало компонентов = невозможно установить на нестандартное железо.
- Отсутствие внешнего доступа = страдают наблюдаемость и возможность подебажить проблемы.

Минусы Kubernetes-дистрибутивов

Идеала не существует. Любые ограничения, помимо безопасности, также и сложности эксплуатации.

- Мало компонентов = невозможно установить на нестандартно
- Отсутствие внешнего доступа = страдают наблюдаемость и возможность подебажить проблемы.

Минусы Kubernetes-дистрибутивов

Идеала не существует. Любые ограничения, помимо безопасности, приносят также и сложности эксплуатации.

- Мало компонентов = невозможно установить на нестандартное железо.
- Отсутствие внешнего доступа = страдают наблюдаемость и возможность подебажить проблемы.
- Предустановлены компоненты K8s = трудозатратно без пересборки дистрибутива применить какие-то свои наработки.

Совместимость с крупными ИТ-системами

DUC meetup/4

Совместимость с крупными ИТ-системами

Как показывает практика, Kubernetes-дистрибутивы не всегда подходят для крупных отечественных ИТ-решений.

Совместимость с крупными ИТ-системами

Как показывает практика, Kubernetes-дистрибутивы не всегда подходят для крупных отечественных ИТ-решений.

Почему же так получилось?

Этому есть несколько причин.

Совместимость с крупными ИТ-системами

- **Импортозамещение.** Во многих критических отраслях требуются сертифицированные решения, Kubernetes-native-дистрибутивы к ним не относятся.

Совместимость с крупными ИТ-системами

- **Импортозамещение.** Во многих критических отраслях требуются сертифицированные решения, Kubernetes-native-дистрибутивы к ним не относятся.
- **Приобретённое ПО.** У компании уже есть закупленные дистрибутивы.

Совместимость с крупными ИТ-системами

- **Импортозамещение.** Во многих критических отраслях требуются сертифицированные решения, Kubernetes-native-дистрибутивы к ним не относятся.
- **Приобретённое ПО.** У компании уже есть закупленные дистрибутивы.
- **Поддержка облаками.** Ни публичные, ни приватные отечественные облака не предоставляют поддержку Kubernetes-native-дистрибутивов «из коробки».

Совместимость с крупными ИТ-системами

- **Импортозамещение.** Во многих критических отраслях требуются сертифицированные решения, Kubernetes-native-дистрибутивы к ним не относятся.
- **Приобретённое ПО.** У компании уже есть закупленные дистрибутивы.
- **Поддержка облаками.** Ни публичные, ни приватные отечественные облака не предоставляют поддержку Kubernetes-native-дистрибутивов «из коробки».
- **Собственная экспертиза.** Любая крупная организация имеет устоявшиеся практики.

Совместимость с крупными ИТ-системами

- **Bare metal.** Часто крупные организации используют не облачные инсталляции, а bare metal. Поэтому парадигма «перезаказать узел, а не разбираться, что произошло» в данном случае становится неприменимой. Также эта практика плохо подходит для stateful-приложений.

Совместимость с крупными ИТ-системами

- **Bare metal.** Часто крупные организации используют не облачные инсталляции, а bare metal. Поэтому парадигма «перезаказать узел, а не разбираться, что произошло» в данном случае становится неприменимой. Также эта практика плохо подходит для stateful-приложений.
- **Нестандартное железо.** Применение bare-metal-инсталляций зачастую сопровождается применением нестандартного железа.

Совместимость с крупными ИТ-системами

- **Bare metal.** Часто крупные организации используют не облачные инсталляции, а bare metal. Поэтому парадигма «перезаказать узел, а не разбираться, что произошло» в данном случае становится неприменимой. Также эта практика плохо подходит для stateful-приложений.
- **Нестандартное железо.** Применение bare-metal-инсталляций зачастую сопровождается применением нестандартного железа.
- **Отсутствие поддержки.** Иностранные компании не предоставляют поддержку для своих решений на территории РФ.

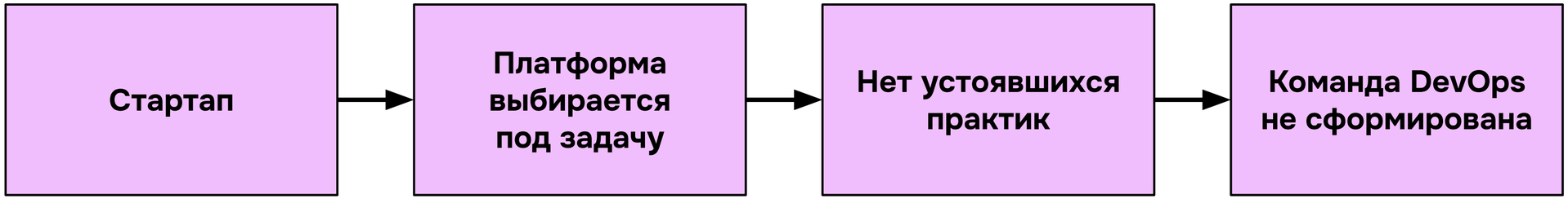
Совместимость с крупными ИТ-системами

- **Bare metal.** Часто крупные организации используют не облачные инсталляции, а bare metal. Поэтому парадигма «перезаказать узел, а не разбираться, что произошло» в данном случае становится неприменимой. Также эта практика плохо подходит для stateful-приложений.
- **Нестандартное железо.** Применение bare-metal-инсталляций зачастую сопровождается применением нестандартного железа.
- **Отсутствие поддержки.** Иностранные компании не предоставляют поддержку для своих решений на территории РФ.
- **Сложность установки дополнительного и нестандартного ПО.** Сложно применять сценарий установки ПО не в контейнеры, а на сами узлы.

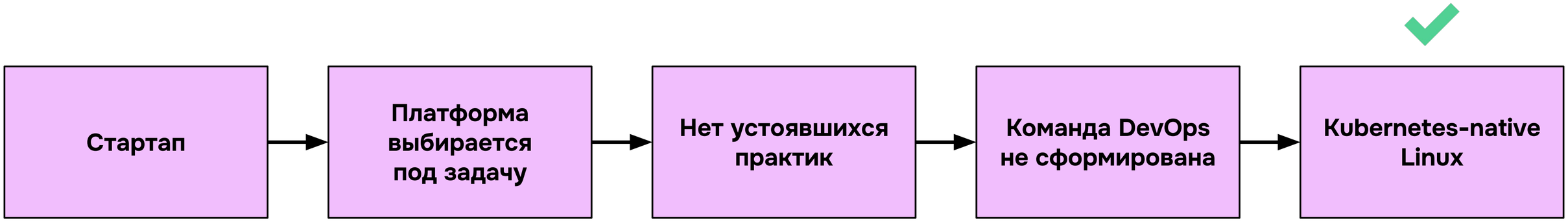
При выборе решения для развёртывания Kubernetes недостаточно опираться только на его признание в сообществе. **На выбор влияют:**

- доступное компании «железо»;
- требования ИБ и регуляторов;
- устоявшиеся практики;
- количество инженеров;
- другие факторы.

Стартап


Стартап
Платформа выбирается под задачу
Нет устоявшихся практик
Команда DevOps не сформирована

Стартап / новое направление



Production-проект

Production-проект

Как использовать любой дистрибутив для нужд Kubernetes?

DUC meetup/4

Как использовать любой дистрибутив для нужд Kubernetes?

Чтобы можно было научиться использовать любой дистрибутив как базу для установки и настройки узла кластера Kubernetes, нужно выполнить несколько условий.

Как использовать любой дистрибутив для нужд Kubernetes?

- Нужно научиться устанавливать на узел необходимое ПО (компоненты Kubernetes, CRI, CNI, ...).

Как использовать любой дистрибутив для нужд Kubernetes?

- Нужно научиться устанавливать на узел необходимое ПО (компоненты Kubernetes, CRI, CNI, ...).
- Нужно научиться настраивать ПО на узле под нужды Kubernetes.

Как использовать любой дистрибутив для нужд Kubernetes?

- Нужно научиться устанавливать на узел необходимое ПО (компоненты Kubernetes, CRI, CNI, ...).
- Нужно научиться настраивать ПО на узле под нужды Kubernetes.
- Нужна reconcile-машина, которая будет поддерживать наше ПО и наши настройки в актуальном состоянии.

Как использовать любой дистрибутив для нужд Kubernetes?

- Нужно научиться устанавливать на узел необходимое ПО (компоненты Kubernetes, CRI, CNI, ...).
- Нужно научиться настраивать ПО на узле под нужды Kubernetes.
- Нужна reconcile-машина, которая будет поддерживать наше ПО и наши настройки в актуальном состоянии.
- Вмешательство в дистрибутив должно быть минимальным.

Как использовать любой дистрибутив для нужд Kubernetes?

- Нужно научиться устанавливать на узел необходимое ПО (компоненты Kubernetes, CRI, CNI, ...).

Поскольку разные дистрибутивы используют разные наборы библиотек для динамической линковки, мы должны поставлять своё ПО либо собранным статически, либо вместе с библиотеками, с которыми оно слинковано.

Как использовать любой дистрибутив для нужд Kubernetes?

- Нужно научиться устанавливать на узел необходимое ПО (компоненты Kubernetes, CRI, CNI, ...).

Способ установки ПО на узлы должен быть не привязанным к пакетному менеджеру конкретного дистрибутива. Так как мы работаем с Kubernetes, основной источник правды для нас — registry.

Пакеты, которые мы ставим на узел, хранятся в registry. Установка производится по предварительно выбранному пути (в нашем случае /opt/deckhouse). Абсолютно все компоненты на узле, необходимые на узле, ставятся именно таким образом.

Как использовать любой дистрибутив для нужд Kubernetes?

- Нужно научиться устанавливать на узел необходимое ПО (компоненты Kubernetes, CRI, CNI, ...).

Абсолютно все бинарные файлы, устанавливаемые на узел, содержат подписи. Также присутствует контроль целостности во время выполнения. Более подробно о работе контроля целостности можно узнать из доклада моего коллеги Максима Набоких на Highload++

Как использовать любой дистрибутив для нужд Kubernetes?

- Нужно научиться настраивать ПО на узле под нужды Kubernetes.

В нашей реализации за эту часть отвечают два компонента:

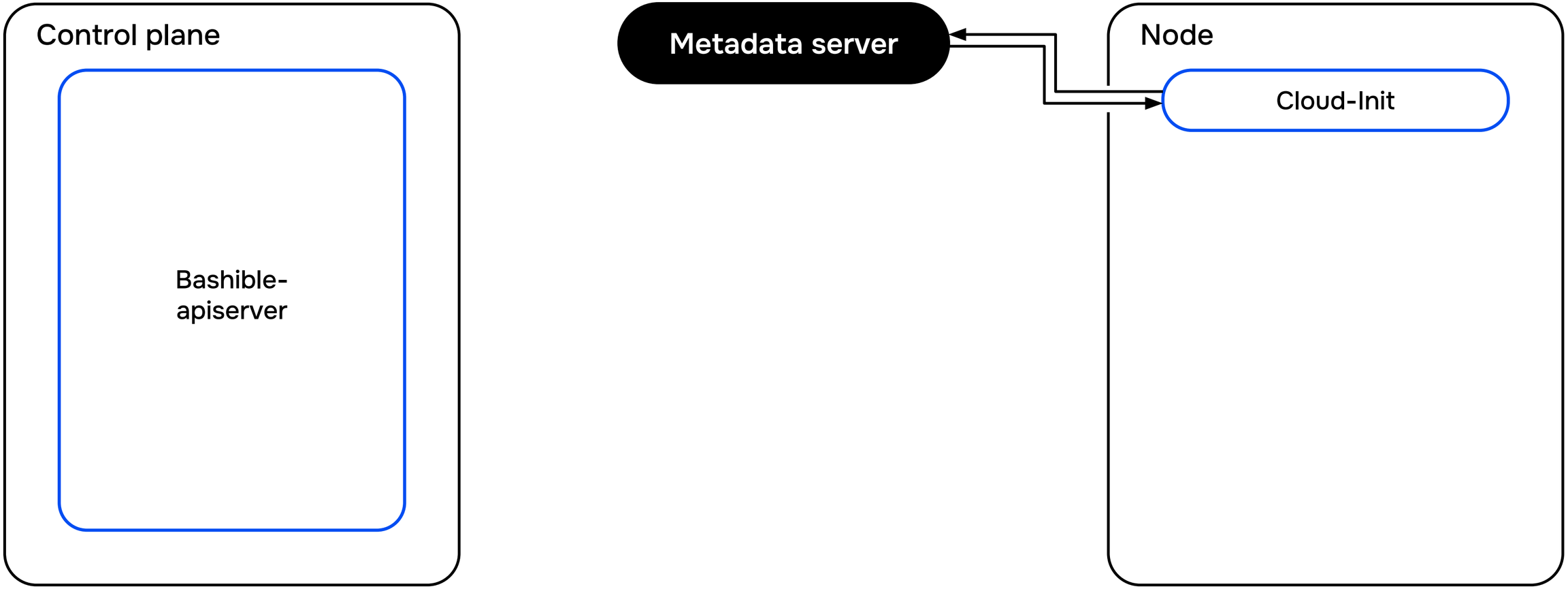
- **Bashible** — фреймворк, который работает на узле.
- **Bashible-apiserver** — движок рендера темплейтов для узлов и компонент для передачи срендеренных скриптов на узлы.

Как использовать любой дистрибутив для нужд Kubernetes?

Подобный подход использует OpenShift — там есть специальный Machine Config Operator, обеспечивающий применение конфигурации на узлах.

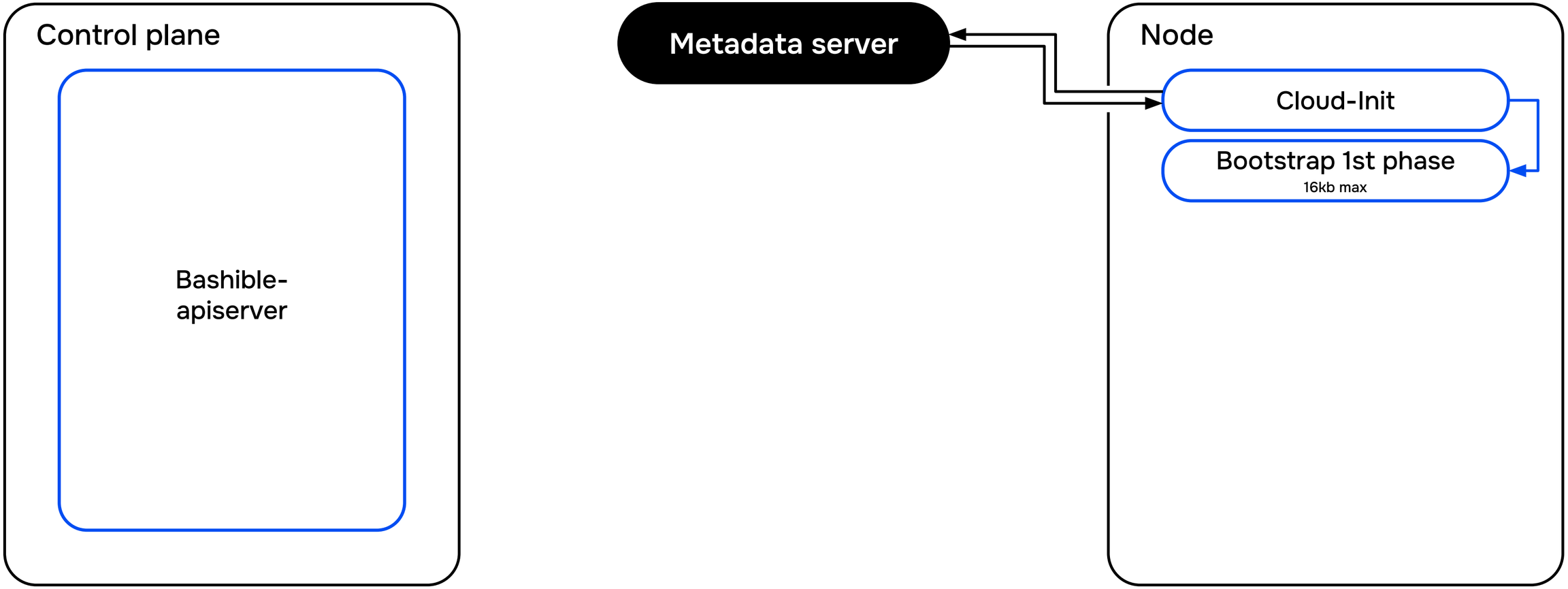
Как использовать любой дистрибутив для нужд Kubernetes?

- Нужно научиться настраивать ПО на узле под нужды Kubernetes.



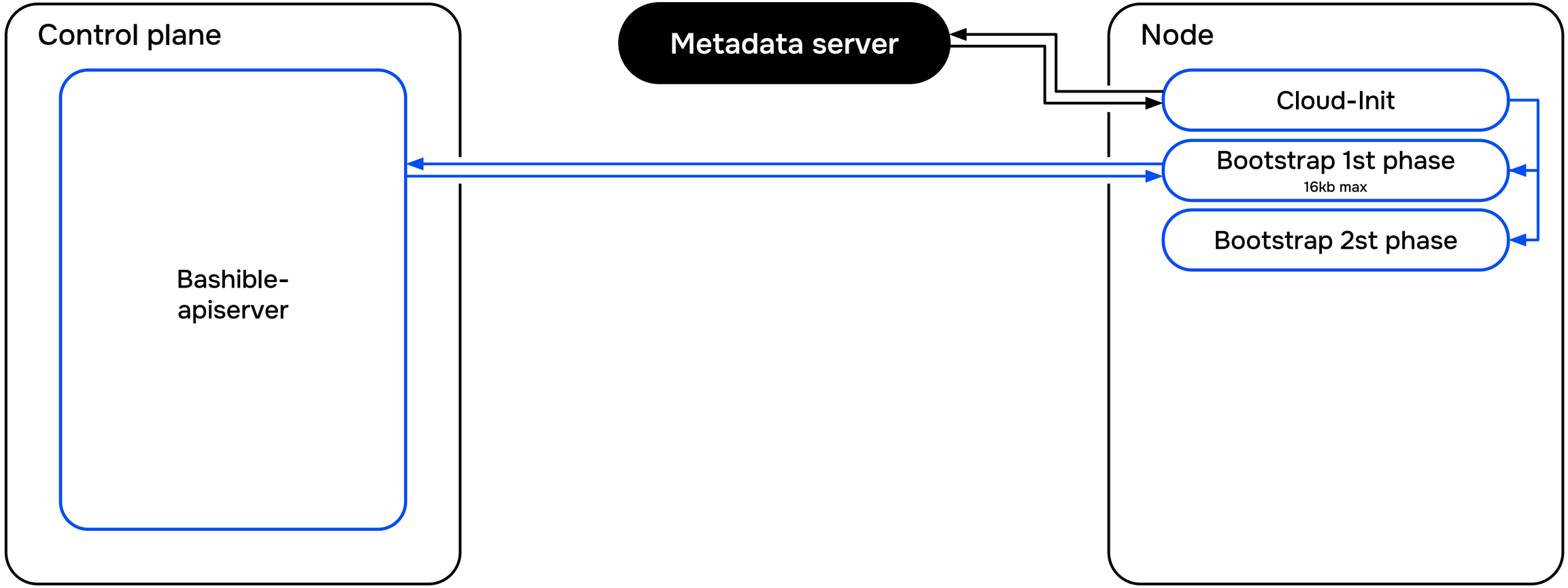
Как использовать любой дистрибутив для нужд Kubernetes?

- Нужно научиться настраивать ПО на узле под нужды Kubernetes.



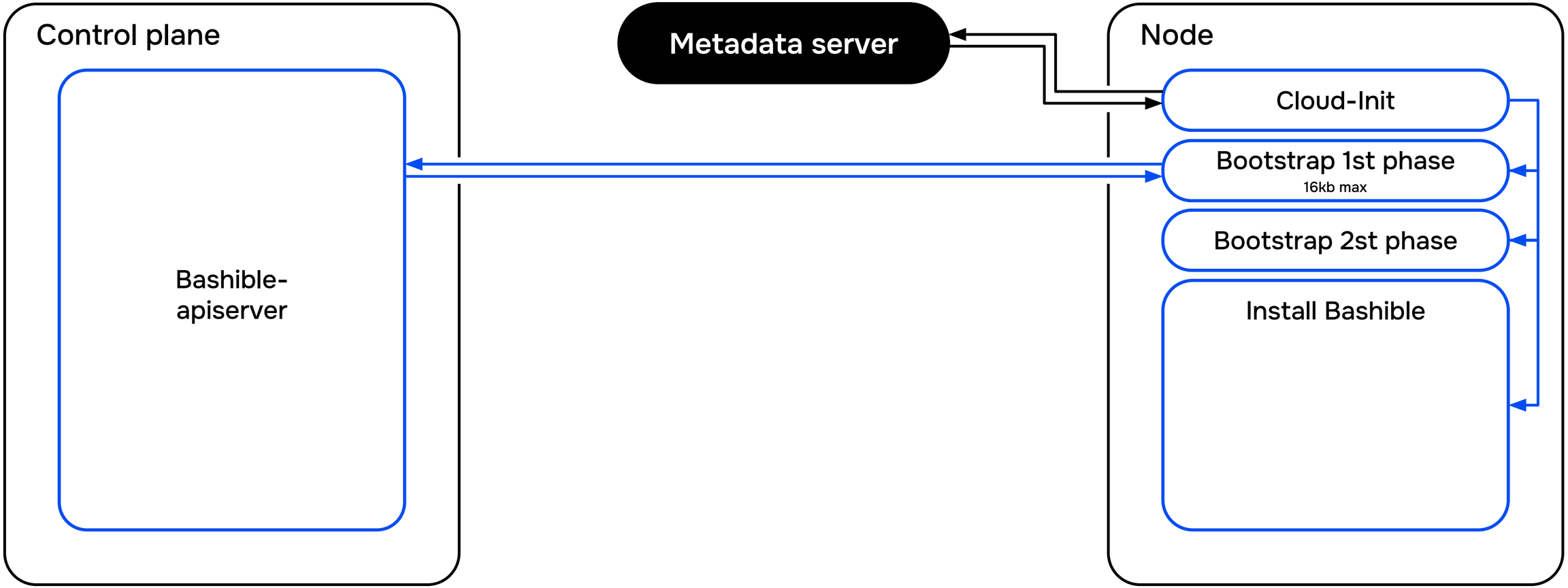
Как использовать любой дистрибутив для нужд Kubernetes?

- Нужно научиться настраивать ПО на узле под нужды Kubernetes.



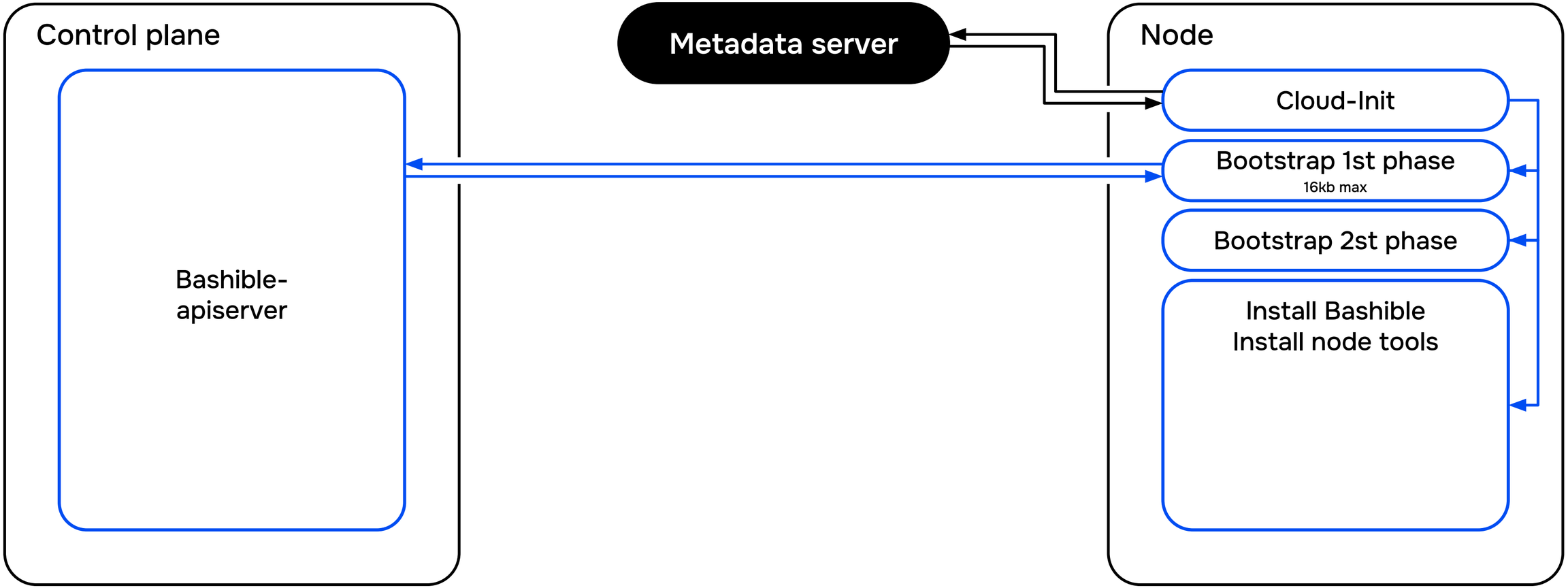
Как использовать любой дистрибутив для нужд Kubernetes?

- Нужно научиться настраивать ПО на узле под нужды Kubernetes.



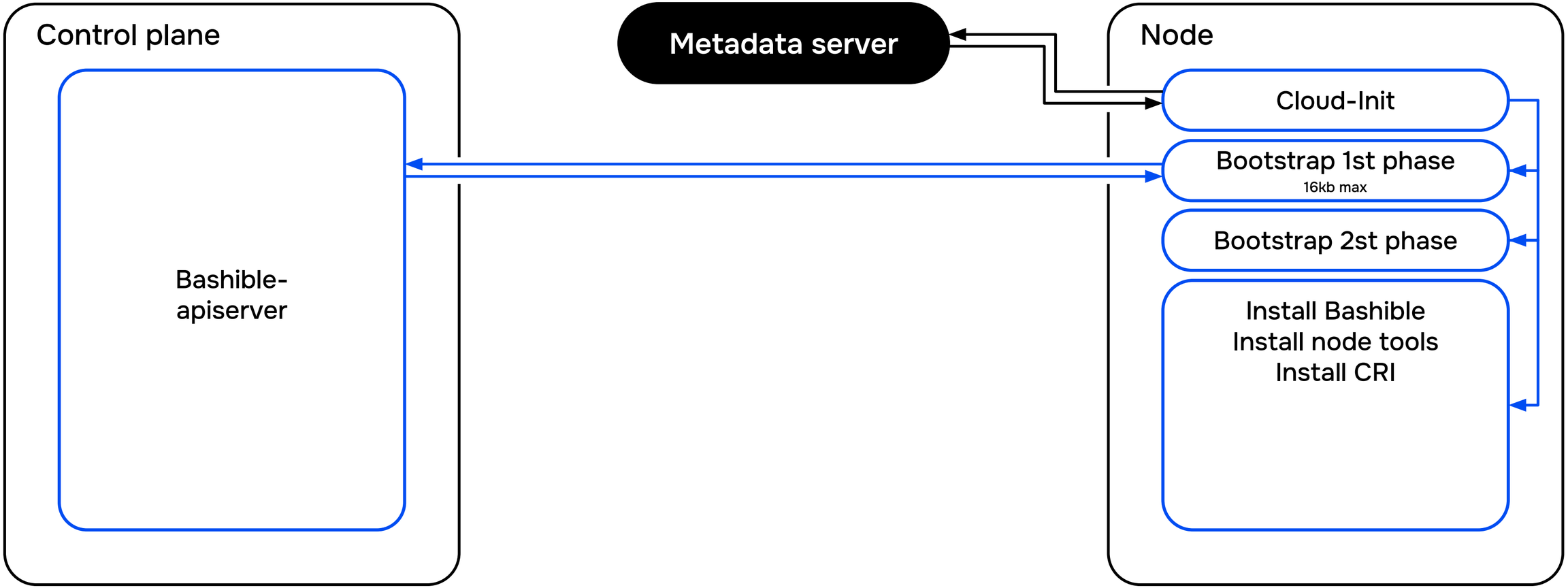
Как использовать любой дистрибутив для нужд Kubernetes?

- Нужно научиться настраивать ПО на узле под нужды Kubernetes.



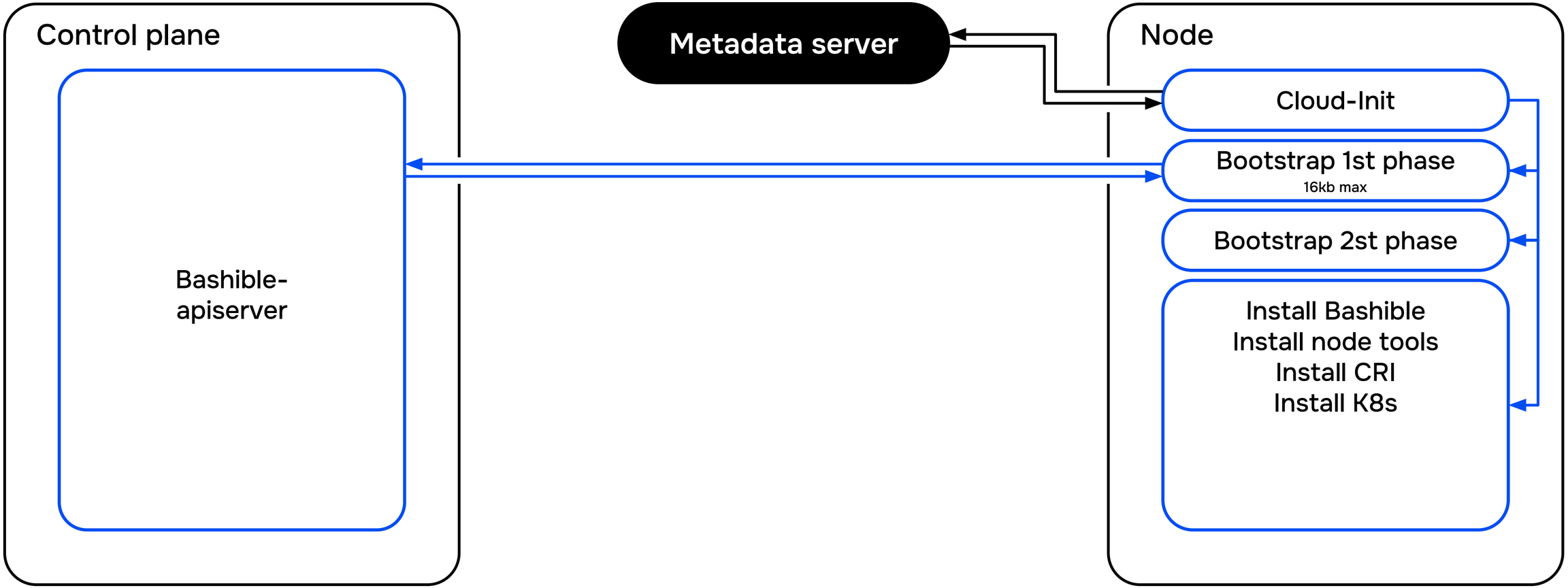
Как использовать любой дистрибутив для нужд Kubernetes?

- Нужно научиться настраивать ПО на узле под нужды Kubernetes.



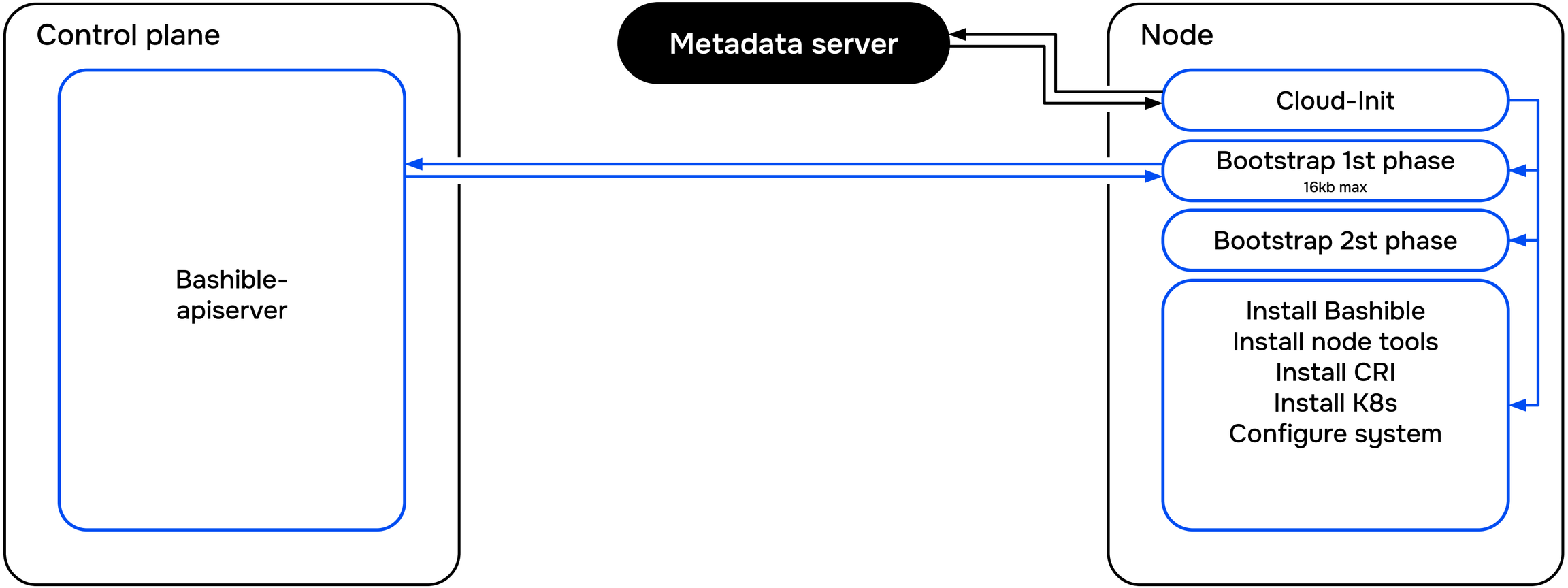
Как использовать любой дистрибутив для нужд Kubernetes?

- Нужно научиться настраивать ПО на узле под нужды Kubernetes.



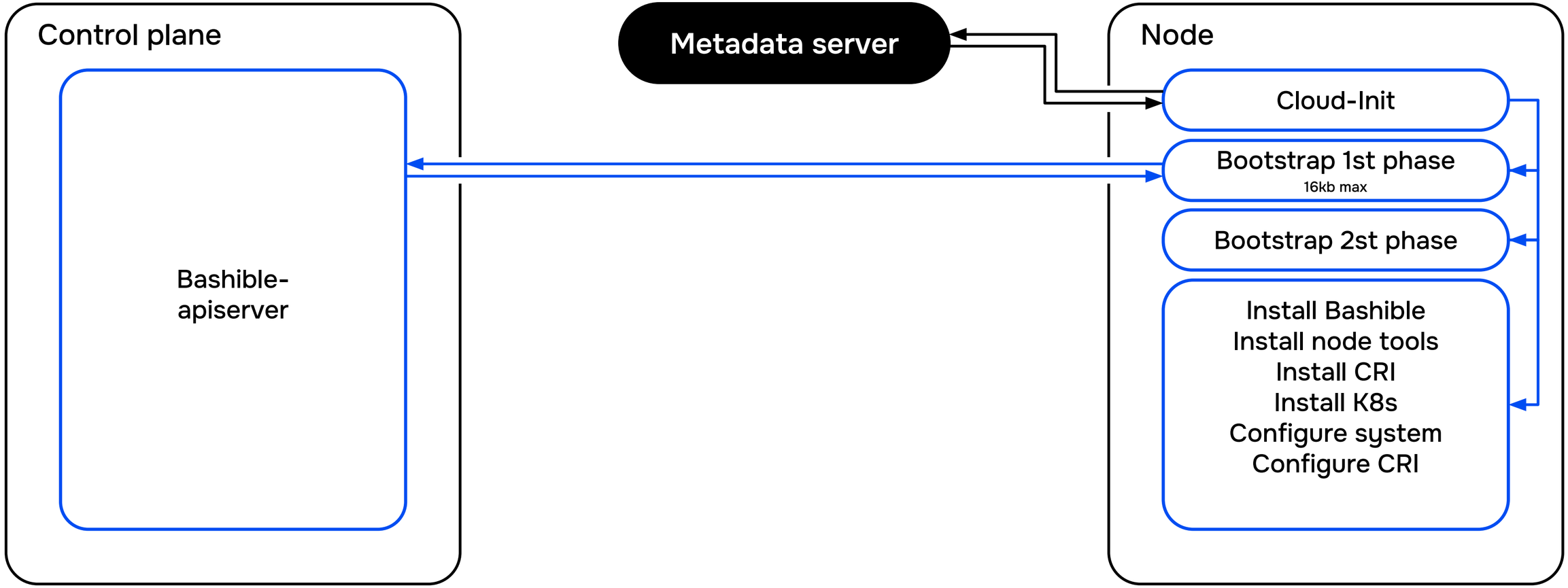
Как использовать любой дистрибутив для нужд Kubernetes?

- Нужно научиться настраивать ПО на узле под нужды Kubernetes.



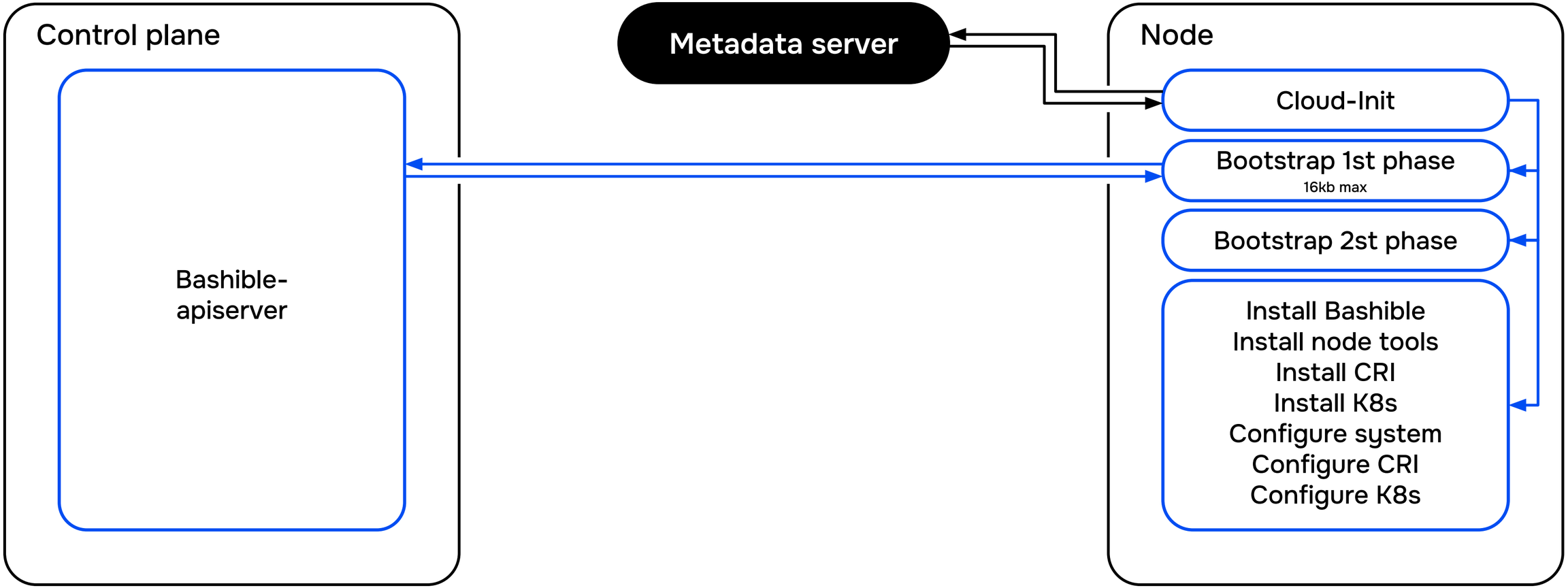
Как использовать любой дистрибутив для нужд Kubernetes?

- Нужно научиться настраивать ПО на узле под нужды Kubernetes.



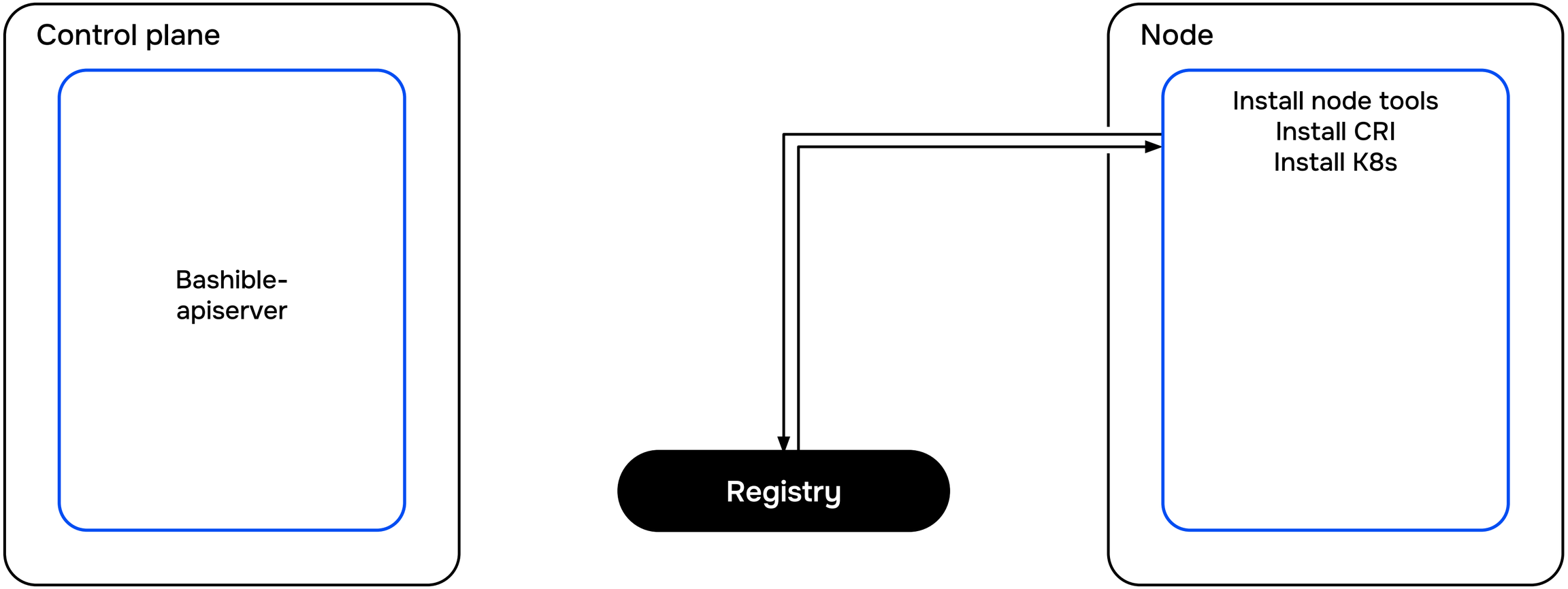
Как использовать любой дистрибутив для нужд Kubernetes?

- Нужно научиться настраивать ПО на узле под нужды Kubernetes.



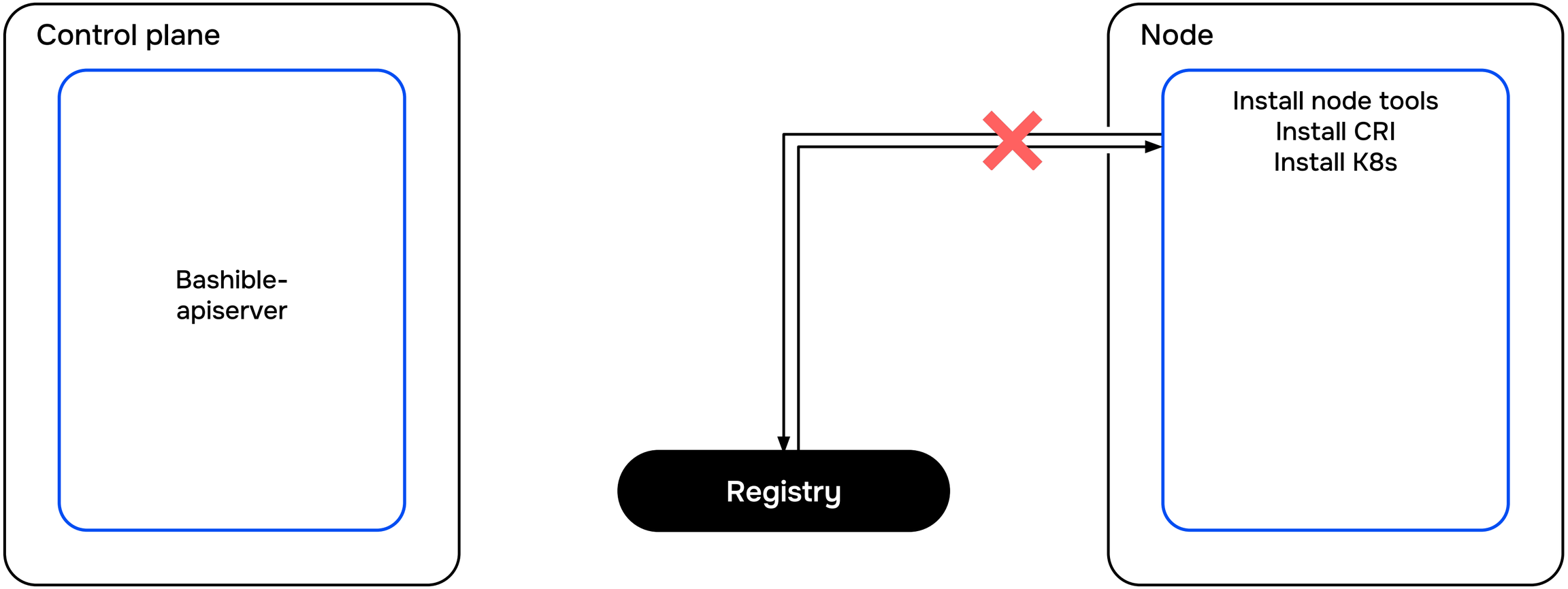
Как использовать любой дистрибутив для нужд Kubernetes?

- Нужно научиться настраивать ПО на узле под нужды Kubernetes.



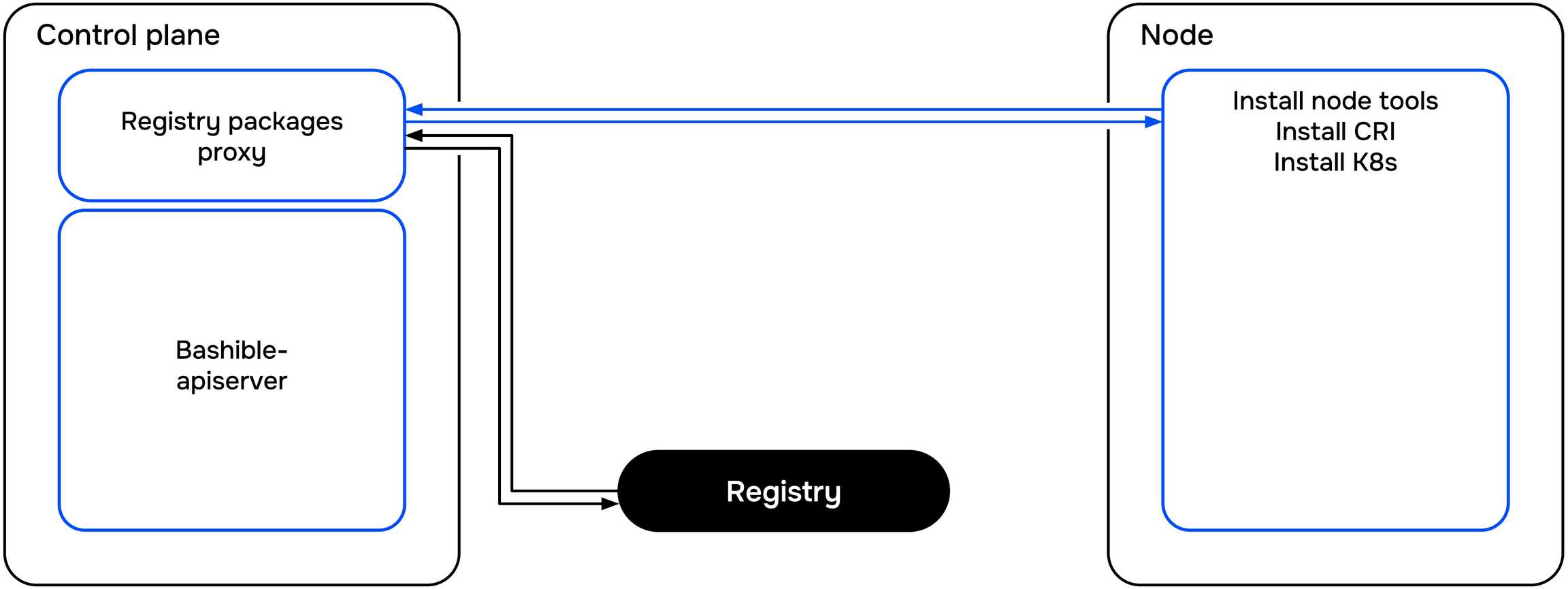
Как использовать любой дистрибутив для нужд Kubernetes?

- Нужно научиться настраивать ПО на узле под нужды Kubernetes.



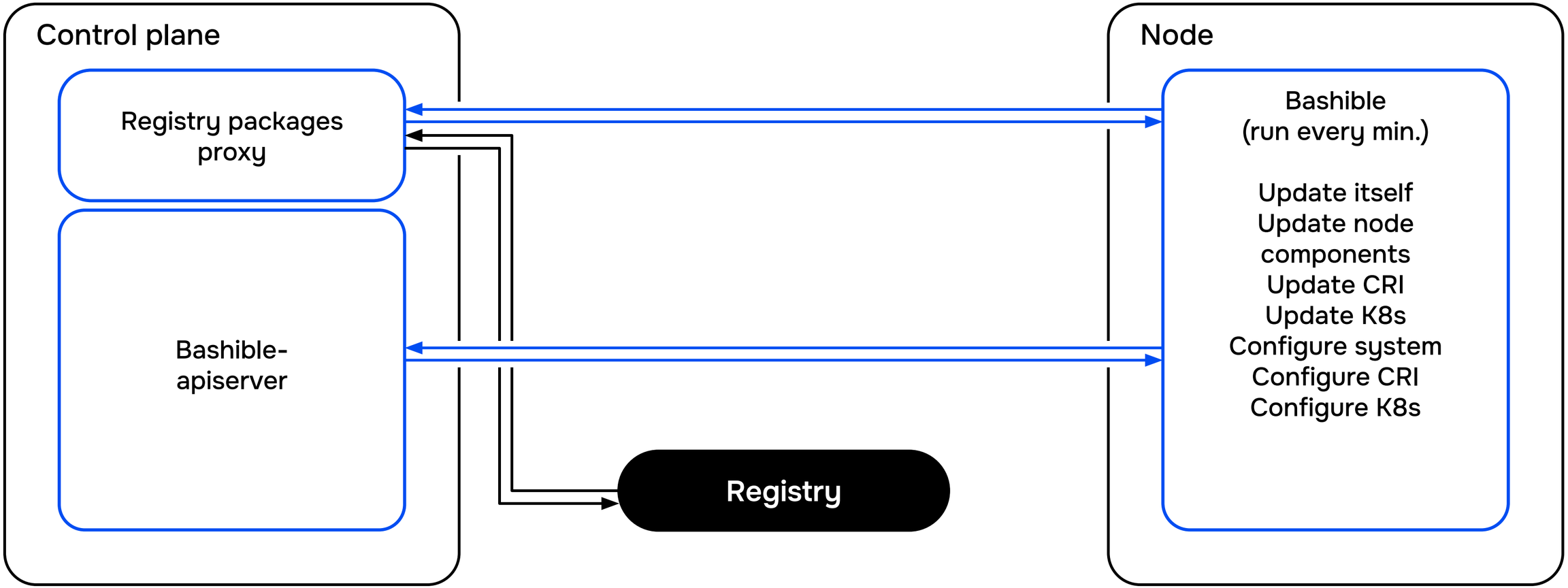
Как использовать любой дистрибутив для нужд Kubernetes?

- Нужно научиться настраивать ПО на узле под нужды Kubernetes.



Как использовать любой дистрибутив для нужд Kubernetes?

- Нужна reconcile-машина, которая будет поддерживать наше ПО и наши настройки в актуальном состоянии.



Как использовать любой дистрибутив для нужд Kubernetes?

- Единая точка правды — registry.

Как использовать любой дистрибутив для нужд Kubernetes?

- Единая точка правды — registry.
- Все ПО, которое ставится на узел, защищено на всех этапах цифровыми подписями и контролем целостности.

Как использовать любой дистрибутив для нужд Kubernetes?

- Единая точка правды — registry.
- Все ПО, которое ставится на узел, защищено на всех этапах цифровыми подписями и контролем целостности.
- ПО не содержит внешних зависимостей, поэтому применимо на любом дистрибутиве.

Как использовать любой дистрибутив для нужд Kubernetes?

- Единая точка правды — registry.
- Все ПО, которое ставится на узел, защищено на всех этапах цифровыми подписями и контролем целостности.
- ПО не содержит внешних зависимостей, поэтому применимо на любом дистрибутиве.
- ПО ставится по отдельному пути, не пересекающемуся с системным.

Как использовать любой дистрибутив для нужд Kubernetes?

- Единая точка правды — registry.
- Все ПО, которое ставится на узел, защищено на всех этапах цифровыми подписями и контролем целостности.
- ПО не содержит внешних зависимостей, поэтому применимо на любом дистрибутиве.
- ПО ставится по отдельному пути, не пересекающемуся с системным.
- Все systemd-юниты имеют префикс d8-*.

Как использовать любой дистрибутив для нужд Kubernetes?

- Единая точка правды — registry.
- Все ПО, которое ставится на узел, защищено на всех этапах цифровыми подписями и контролем целостности.
- ПО не содержит внешних зависимостей, поэтому применимо на любом дистрибутиве.
- ПО ставится по отдельному пути, не пересекающемуся с системным.
- Все systemd-юниты имеют префикс d8-*.
- Есть reconcile-машина для поддержания узла в желаемом состоянии.

Как использовать любой дистрибутив для нужд Kubernetes?

- Единая точка правды — registry.
- Все ПО, которое ставится на узел, защищено на всех этапах цифровыми подписями и контролем целостности.
- ПО не содержит внешних зависимостей, поэтому применимо на любом дистрибутиве.
- ПО ставится по отдельному пути, не пересекающемуся с системным.
- Все systemd-юниты имеют префикс d8-*.
- Есть reconcile-машина для поддержания узла в желаемом состоянии.
- SSH-доступ к узлу в большинстве сценариев НЕ НУЖЕН.

Как использовать любой дистрибутив для нужд Kubernetes?

Необходимые зависимости для работы:

- **Python** (закрывается установкой cloud-init).

Как использовать любой дистрибутив для нужд Kubernetes?

Необходимые зависимости для работы:

- **Python** (закрывается установкой cloud-init).
- **Базовые утилиты** (grep, awk, find, bash, coreutils).

Как использовать любой дистрибутив для нужд Kubernetes?

Необходимые зависимости для работы:

- **Python** (закрывается установкой cloud-init).
- **Базовые утилиты** (grep, awk, find, bash, coreutils).
- **SystemD** — система инициализации.

Чем можно управлять?

Чем можно управлять

Bashible-apiserver

```yaml
apiVersion: deckhouse.io/v1
kind: NodeGroup
metadata:
  name: worker
spec:
  cloudInstances:
    classReference:
      kind: DVPInstanceClass
      name: worker
    maxPerZone: 1
    minPerZone: 1
  kubelet:
    containerLogMaxFiles: 4
    containerLogMaxSize: 50Mi
    resourceReservation:
      mode: Auto
  nodeType: CloudEphemeral
```

Чем можно управлять

- Параметры kubelet

Bashible-apiserver

```yaml
apiVersion: deckhouse.io/v1
kind: NodeGroup
metadata:
  name: worker
spec:
  cloudInstances:
    classReference:
      kind: DVPInstanceClass
      name: worker
    maxPerZone: 1
    minPerZone: 1
  kubelet:
    containerLogMaxFiles: 4
    containerLogMaxSize: 50Mi
    resourceReservation:
      mode: Auto
  nodeType: CloudEphemeral
```

Чем можно управлять

- Параметры kubelet
- Параметры CRI

Bashible-apiserver

```yaml
apiVersion: deckhouse.io/v1
kind: NodeGroup
metadata:
  name: worker
spec:
  cloudInstances:
    classReference:
      kind: DVPInstanceClass
      name: worker
    maxPerZone: 1
    minPerZone: 1
  kubelet:
    containerLogMaxFiles: 4
    containerLogMaxSize: 50Mi
    resourceReservation:
      mode: Auto
  nodeType: CloudEphemeral
```

Чем можно управлять

- Параметры kubelet
- Параметры CRI
- Фенсинг

Bashible-apiserver

```yaml
apiVersion: deckhouse.io/v1
kind: NodeGroup
metadata:
  name: worker
spec:
  cloudInstances:
    classReference:
      kind: DVPInstanceClass
      name: worker
    maxPerZone: 1
    minPerZone: 1
  kubelet:
    containerLogMaxFiles: 4
    containerLogMaxSize: 50Mi
    resourceReservation:
      mode: Auto
  nodeType: CloudEphemeral
```

Чем можно управлять

- Параметры kubelet
- Параметры CRI
- Фенсинг
- Параметры GPU

Bashible-apiserver

```yaml
apiVersion: deckhouse.io/v1
kind: NodeGroup
metadata:
  name: worker
spec:
  cloudInstances:
    classReference:
      kind: DVPInstanceClass
      name: worker
    maxPerZone: 1
    minPerZone: 1
  kubelet:
    containerLogMaxFiles: 4
    containerLogMaxSize: 50Mi
    resourceReservation:
      mode: Auto
  nodeType: CloudEphemeral
```

Чем можно управлять

- Параметры kubelet
- Параметры CRI
- Фенсинг
- Параметры GPU
- Параметры апдейта

Bashible-apiserver

```yaml
apiVersion: deckhouse.io/v1
kind: NodeGroup
metadata:
  name: worker
spec:
  cloudInstances:
    classReference:
      kind: DVPInstanceClass
      name: worker
    maxPerZone: 1
    minPerZone: 1
  kubelet:
    containerLogMaxFiles: 4
    containerLogMaxSize: 50Mi
    resourceReservation:
      mode: Auto
  nodeType: CloudEphemeral
```

Чем можно управлять

```yaml
apiVersion: deckhouse.io/v1
kind: NodeUser
metadata:
  name: nodeuser
spec:
  isSudoer: true
  nodeGroups:
  - '*'
  passwordHash: $6$Hash
  sshPublicKeys:
  - ssh-rsa SSH_RSA
  uid: 1100
status:
  errors: {}
```

Bashible-apiserver

```yaml
apiVersion: deckhouse.io/v1
kind: NodeGroup
metadata:
  name: worker
spec:
  cloudInstances:
    classReference:
      kind: DVPInstanceClass
      name: worker
    maxPerZone: 1
    minPerZone: 1
  kubelet:
    containerLogMaxFiles: 4
    containerLogMaxSize: 50Mi
    resourceReservation:
      mode: Auto
  nodeType: CloudEphemeral
```

Чем можно управлять

Позволяет завести пользователя на узлах

```yaml
apiVersion: deckhouse.io/v1
kind: NodeUser
metadata:
  name: nodeuser
spec:
  isSudoer: true
  nodeGroups:
  - '*'
  passwordHash: $6$Hash
  sshPublicKeys:
  - ssh-rsa SSH_RSA
  uid: 1100
status:
  errors: {}
```

Bashible-apiserver

```yaml
apiVersion: deckhouse.io/v1
kind: NodeGroup
metadata:
  name: worker
spec:
  cloudInstances:
    classReference:
      kind: DVPInstanceClass
      name: worker
    maxPerZone: 1
    minPerZone: 1
  kubelet:
    containerLogMaxFiles: 4
    containerLogMaxSize: 50Mi
    resourceReservation:
      mode: Auto
  nodeType: CloudEphemeral
```

Чем можно управлять

```yaml
apiVersion:
deckhouse.io/v1alpha1
kind: NodeGroupConfiguration
metadata:
  name: test.sh
spec:
  bundles:
  - '*'
  content: |
    echo “OK”
  nodeGroups:
  - '*'
  weight: 100
```

Bashible-apiserver

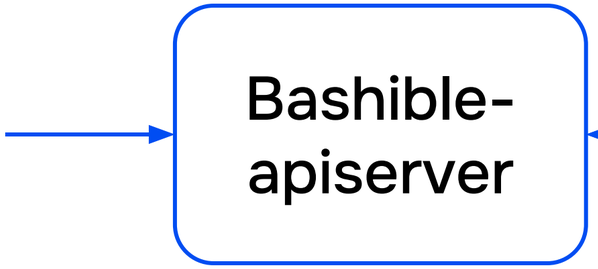
```yaml
apiVersion: deckhouse.io/v1
kind: NodeGroup
metadata:
  name: worker
```

```yaml
apiVersion: deckhouse.io/v1
kind: NodeGroup
metadata:
  name: nodeuser
```

Чем можно управлять

Позволяет выполнить кастомный скрипт на узле, установить стороннее ПО, произвести настройку под нестандартные компоненты

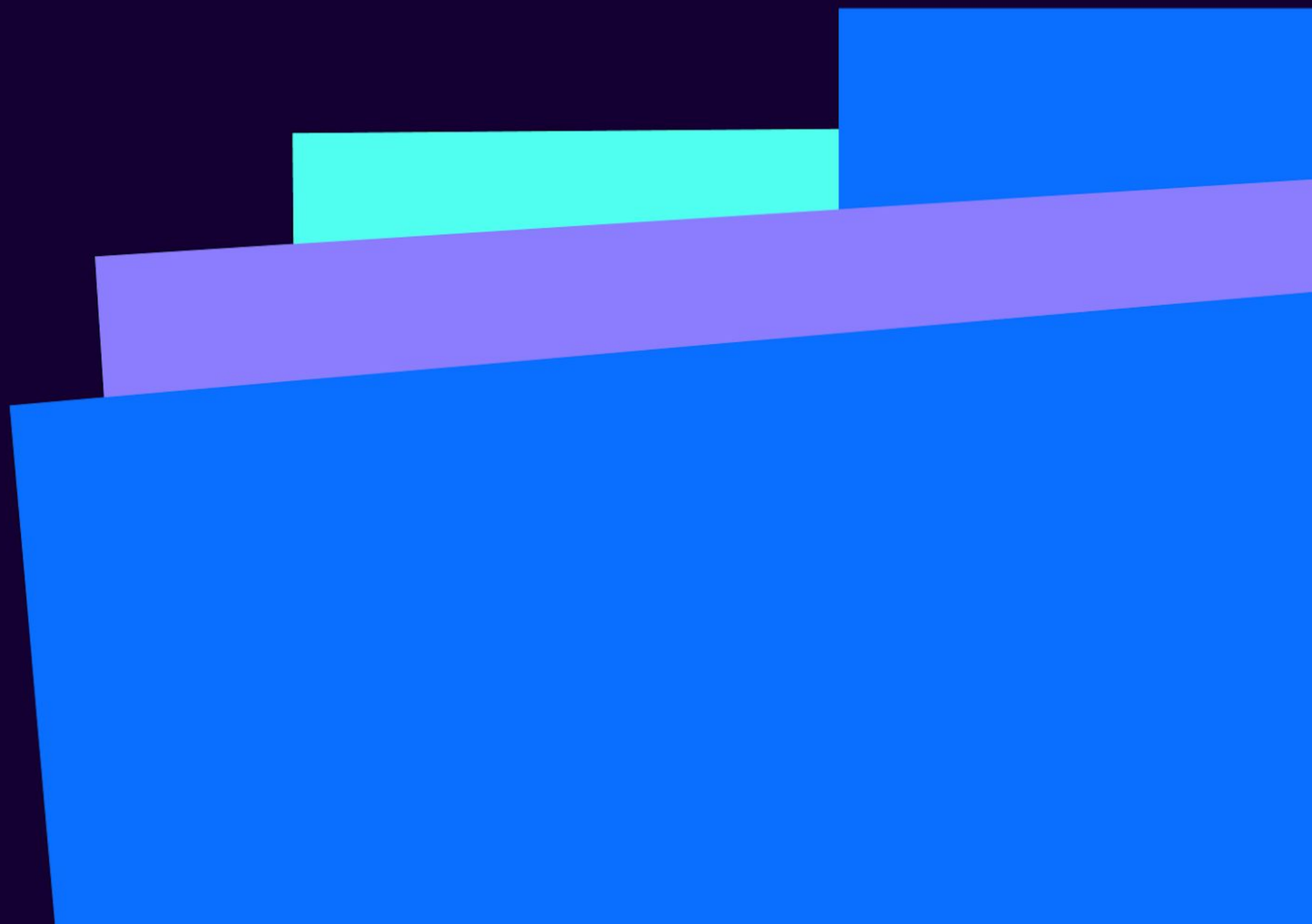
```
apiVersion:
deckhouse.io/v1alpha1
kind: NodeGroupConfiguration
metadata:
  name: test.sh
spec:
  bundles:
  - '*'
  content: |
    echo “OK”
  nodeGroups:
  - '*'
  weight: 100
```




```
apiVersion: deckhouse.io/v1
kind: NodeGroup
metadata:
  name: worker
```

```
apiVersion: deckhouse.io/v1
kind: NodeGroup
metadata:
  name: nodeuser
```

Планы на будущее



DUC meetup/4

Планы на будущее

- Заменить Bashible на агент, написанный на Go.

Планы на будущее

- Заменить Bashible на агент, написанный на Go.
- Убрать зависимость от Python (мини-бинарь для скачивания весом 2 КБ).

Планы на будущее

- Заменить Bashible на агент, написанный на Go.
- Убрать зависимость от Python (мини-бинарь для скачивания весом 2 КБ).
- Свой metadata-server для bare-metal-инсталляций.

Планы на будущее

- Заменить Bashible на агент, написанный на Go.
- Убрать зависимость от Python (мини-бинарь для скачивания весом 2 КБ).
- Свой metadata-server для bare-metal-инсталляций.
- Инсталляция своих утилит в EROFS-слои (доп. контроль).

Планы на будущее

- Заменить Bashible на агент, написанный на Go.
- Убрать зависимость от Python (мини-бинарь для скачивания весом 2 КБ).
- Свой metadata-server для bare-metal-инсталляций.
- Инсталляция своих утилит в EROFS-слои (доп. контроль).
- Запуск своих утилит в выделенном namespace/cgroup.

Выводы

Эволюция. Специализированные Kubernetes-дистрибутивы стали ответом на ограничения традиционных дистрибутивов, обеспечивая оптимизацию под задачи контейнеризации и упрощая управление кластерами.

Эволюция. Специализированные Kubernetes-дистрибутивы стали ответом на ограничения традиционных дистрибутивов, обеспечивая оптимизацию под задачи контейнеризации и упрощая управление кластерами.

Это замечательные решения, имеющие множество плюсов.

Эволюция. Специализированные Kubernetes-дистрибутивы стали ответом на ограничения традиционных дистрибутивов, обеспечивая оптимизацию под задачи контейнеризации и упрощая управление кластерами.

Это замечательные решения, имеющие множество плюсов.

Однако минусы тоже есть: «заточенность» на облака, отсутствие простой кастомизации и замены компонентов «на лету», обновления через перезагрузку.

Не Talos’ом единым. Существуют сценарии, когда применение Talos может быть неоптимальным.

В устоявшихся ИТ-проектах нужно оценивать сложность перехода и подводные камни, связанные с настроенными процессами, bare-metal-инсталляциями, существующим парком ОС, использованием stateful-нагрузок.

Наш подход. Он также имеет свои минусы, использование стандартных дистрибутивов лишает преимуществ Kubernetes-native ОС и требует дополнительных усилий в области ИБ.

Однако же этот подход позволяет плавно интегрироваться с практиками, принятыми в компаниях.

Будущее. Хотелось бы создать дистрибутив, сочетающий простоту и безопасность Kubernetes-native-решений с широкой возможностью кастомизации и замены частей «на лету».

Спасибо за внимание!

DUC meetup/4

Оцените доклад

Денис Романенко
denis.romanenko@flant.com
@RomanenkoDenys

/Следующий доклад

19:50